

꿈임 없는 창의력으로 문제를 해결해 나가는 개발자 유준호입니다.

가치 있는 개발을 지향합니다. 사용자의 요구를 발견하고 파악하여 진정으로 도움이 되는 개발을 하고 싶습니다.

실력 있는 개발자가 되길 원했습니다. 이를 위해 쉬지 않고 개발해 왔습니다. 또한 현재에 안주하지 않고 지속적으로 역량을 키워 나가고 있습니다.

멈춤 없이 문제를 해결해 나가는 개발자가 되고 싶습니다. 그 과정에서 성장하여 정말 멋진 개발을 이어 나가고 싶습니다.



Info

Name	유준호
Birth	1998.02.20
Address	서울시 중랑구 중화동
Phone	010-3792-6395
E-mail	juno980220@naver.com
Git	https://github.com/YooJuno
Youtube	https://youtube.com/@yoojuno

EDUCATIONS

(25.01 ~ 25.10) 삼성 청년 SW 아카데미(SSAFY) / 임베디드 로봇 트랙(광주)

- [Web] Vue.js, Node.js, MySQL 학습 및 웹프로젝트 수행
- [Robot] 기구학 및 로봇팔 제어 학습 및 프로젝트 수행
- [Project] Git, Jira를 활용한 기획 및 개발 수행

(18.02 ~ 24.08) 한동대학교 / AI 컴퓨터공학 심화 / 졸업

EXPERIENCES

(24.07 ~ 24.09) GMD SOFT / 개발 1팀 / 인턴

(22.07 ~ 24.06) 한동대학교 / CGV LAB / 학부 연구생

(23.09 ~ 23.12) 자이트론 / 개발팀 / 인턴

AWARDS

(2023. 05) 캡스톤 디자인 페스티벌 / 우수상 / 한동대학교

Skills

Computer Vision

OpenCV	■■■■■□
Isaac ROS VSLAM	■■■□□
RTAB-MAP	■■■□□
ORB-SLAM3	■■■□□
SfM (COLMAP)	■■■□□
LiDAR Clustering	■■■□□

Embedded

STM32	■■■■■
ESP32	■■■■■
Jetson Orin Nano	■■■■■□
Raspberry Pi	■■■■■□
nRF52840	■■■□□
(with UART, I2C, SPI, DMA, PWM)	

Network

UDP/TCP	■■■■■□
MQTT	■■■■■□
gRPC	■■■□□
Boost ASIO (C++)	■■□□□

Robotics

ROS 1/2	■■■■■
RoboDK	■■■□□
Nav2	■■■□□
Dobot Magician	■■■□□
기구학	■■□□□

OS

Linux	■■■■■□
-------	--------

Web

Vue.JS	■■■□□
Node.JS	■■■□□
MySQL	■■■□□
WebRTC	■■□□□

2025.07 ~ 2025.08 (6인)

한강 자율 주행 배달 로봇

■ 프로젝트 개요

Git : 보안상 공개 불가

목표 : 야외 환경에서의 카메라 기반 자율 주행 배달 서비스 제공

배경 : 기존 한강에서 배달 음식 수령을 위해 먼 거리를 걸어가야함.

주요 기능 : 자율 주행, 사용자 얼굴 인식, 실시간 스트리밍, 다중 배달

■ 담당 역할 및 기여

역할 : 팀장, Project Manager, Robotics.

기여 : 자율 주행 로봇 개발.

■ 사용 기술

ROS2 : 기능 단위의 노드 통합 관리

MQTT : Spring Boot 서버와의 통신 수행

Isaac-VSLAM : 카메라 기반 위치 추정 및 지도 작성(젯슨보드에서 실행)

FaceNet : 얼굴 이미지 비교 딥러닝.

Pure-Pursuit : 출발지 -> 고객의 위치 까지의 경로 추종.

GPS <-> SLAM : GPS 좌표와 SLAM좌표 실시간 변환 수행.

Docker : Visual-SLAM 환경 관리

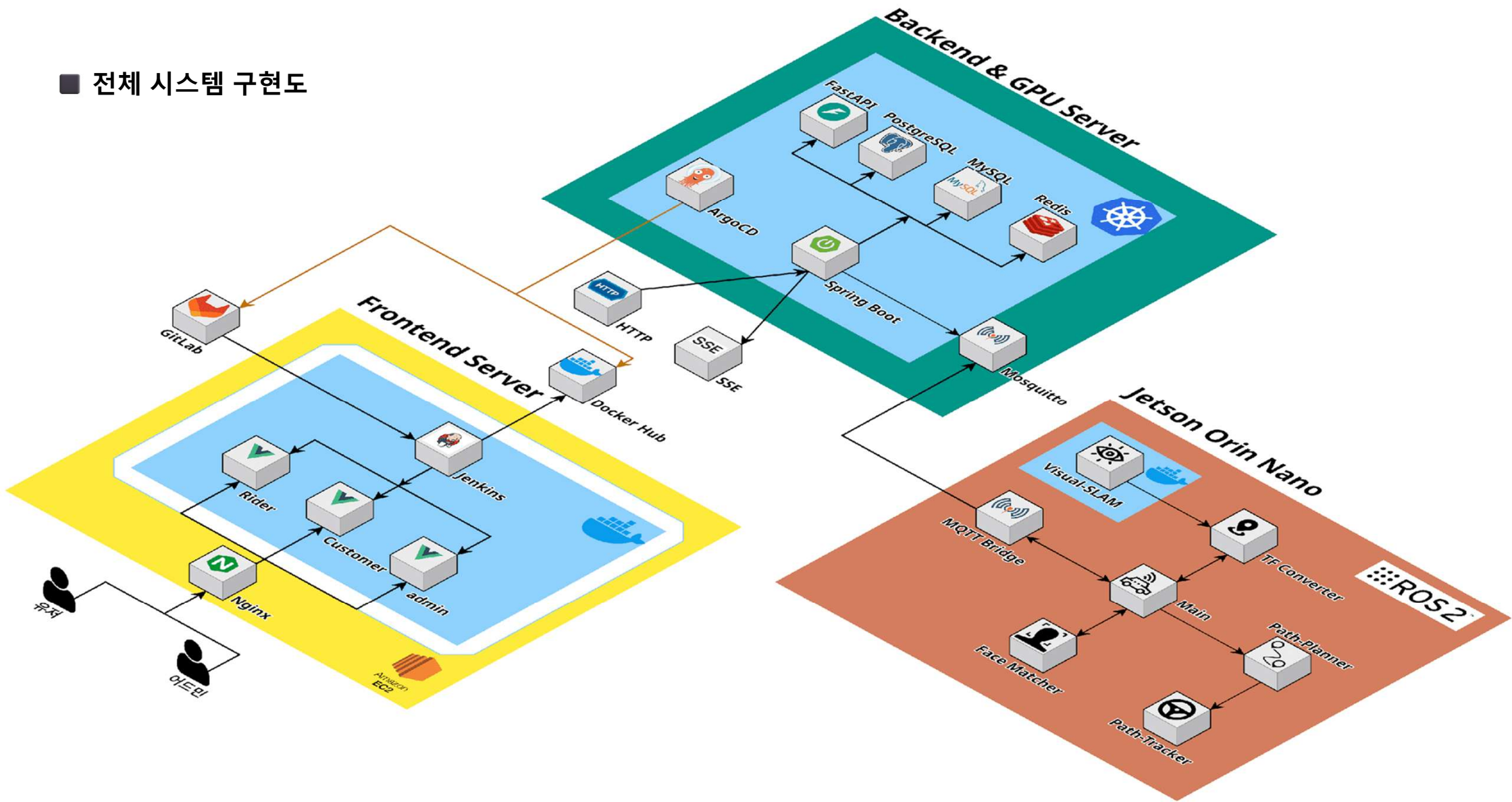
■ 협업 툴

Jira, GitLab(Mono Repo)



[사진 1] 자율 주행 배달 로봇 LiNKY 정면

■ 전체 시스템 구현도

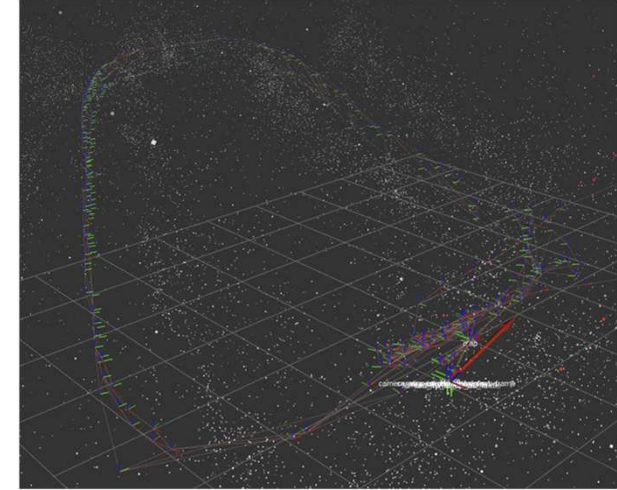


[사진 2] System Architecture

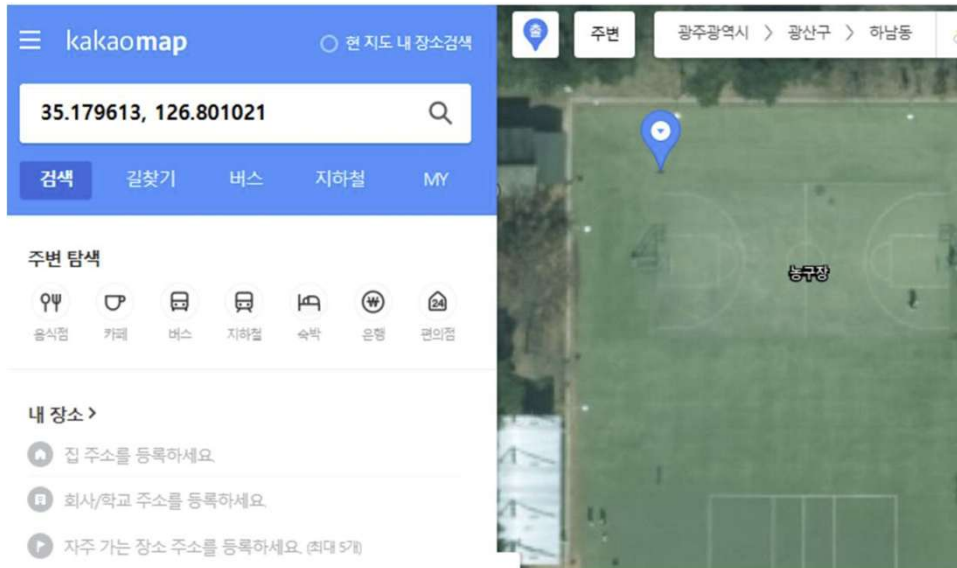
한강 자율 주행 배달 로봇

■ 위치 추정 (Isaac ROS Visual-SLAM)

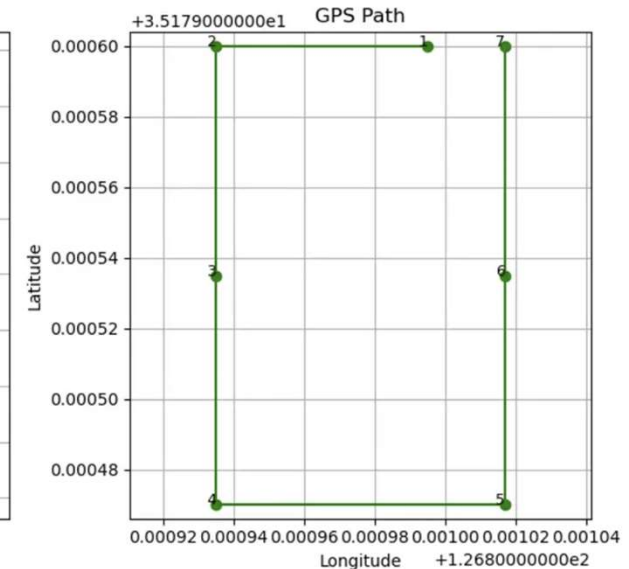
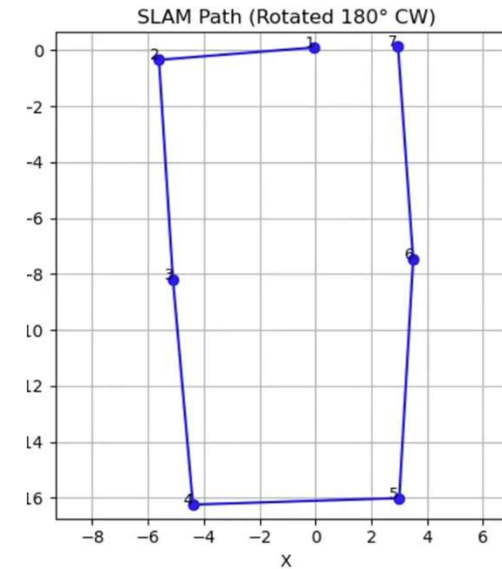
- Jetson Orin Nano 에 최적화된 VSLAM 모델인 Isaac ROS Visual-SLAM을 사용하여 온디바이스 기반 위치 추론 수행.
- Realsense D435i Camera를 사용하여 보다 정밀한 Point Cloud 획득.
- 주행 환경에 대한 지도를 사전에 생성하여 SLAM 좌표 정보 획득.
- 주행환경에 대한 GPS 정보 또한 습득하여 SLAM 좌표와 맵핑.
- 이를 통해 사용자의 GPS 좌표를 SLAM 좌표로 실시간 변환하여 사용자의 위치로 자율 주행 수행.
- 차량의 SLAM 좌표를 GPS 좌표로 변환하여 고객 인터페이스에서 차량의 좌표 실시간 확인.



[사진 3] 생성한 3D Visual-SLAM Point Cloud



[사진 4] 주행 경로 GPS 데이터 획득

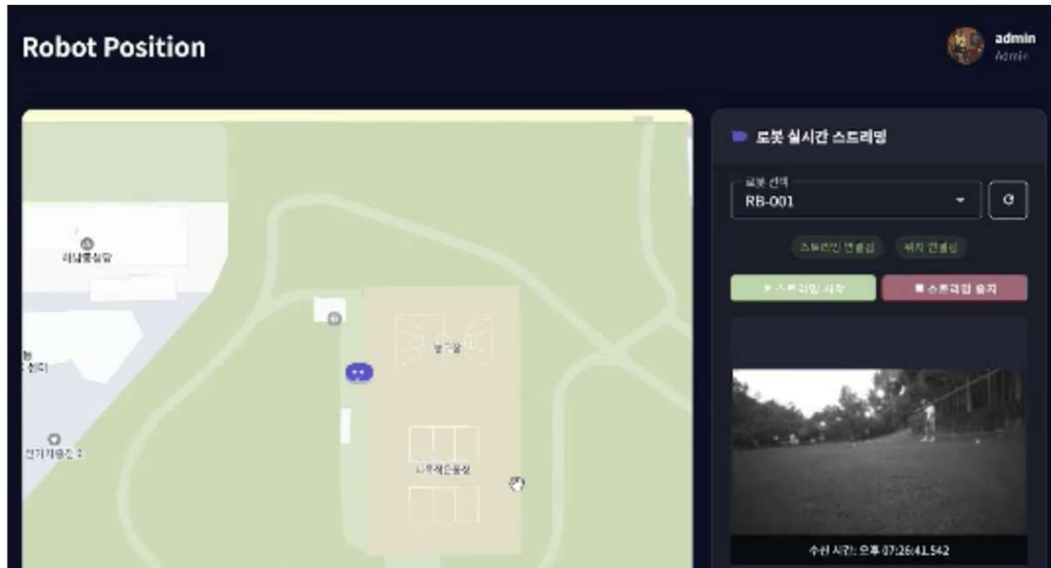


[사진 5] SLAM & GPS Translation and Rotation

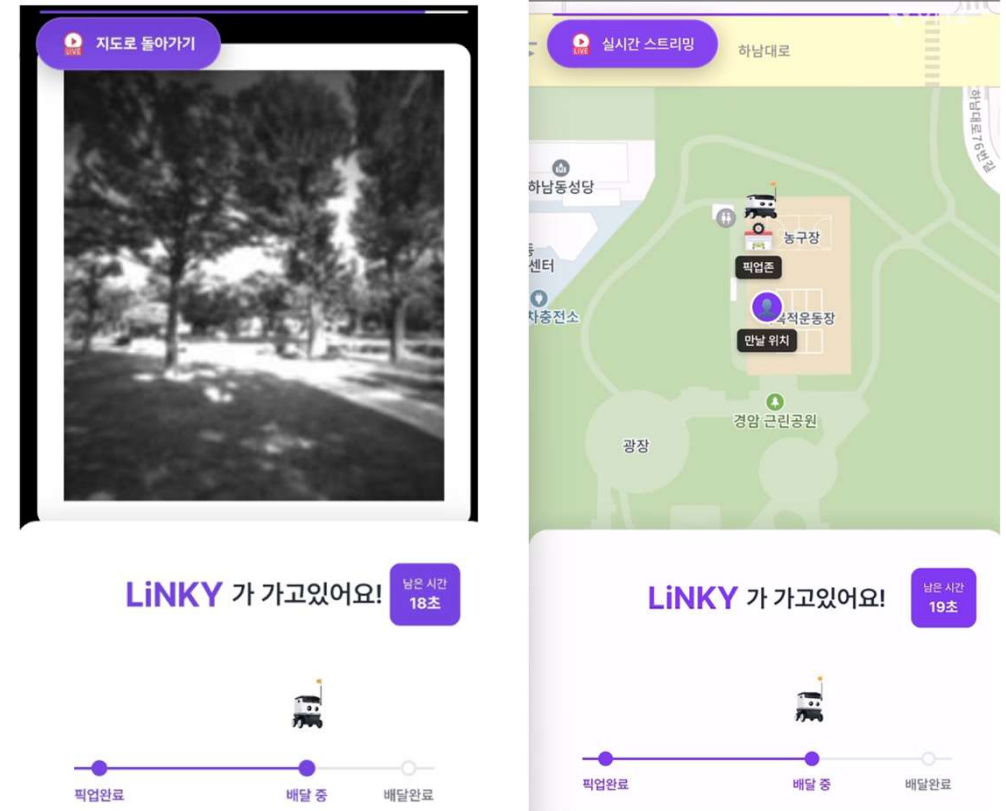
한강 자율 주행 배달 로봇

■ 실시간 위치 및 촬영 영상 스트리밍 (with MQTT)

- SLAM <-> GPS 변환을 통해 구한 실시간 GPS 좌표를 서버로 송신.
- 실시간 입력 이미지를 base64 형식으로 변환하여 서버로 송신.
- 송신한 정보는 백엔드 서버를 거쳐 관리자 및 고객 페이지에서 실시간 확인 가능.



[사진 6] 관리자 UI (로봇 정보 확인)

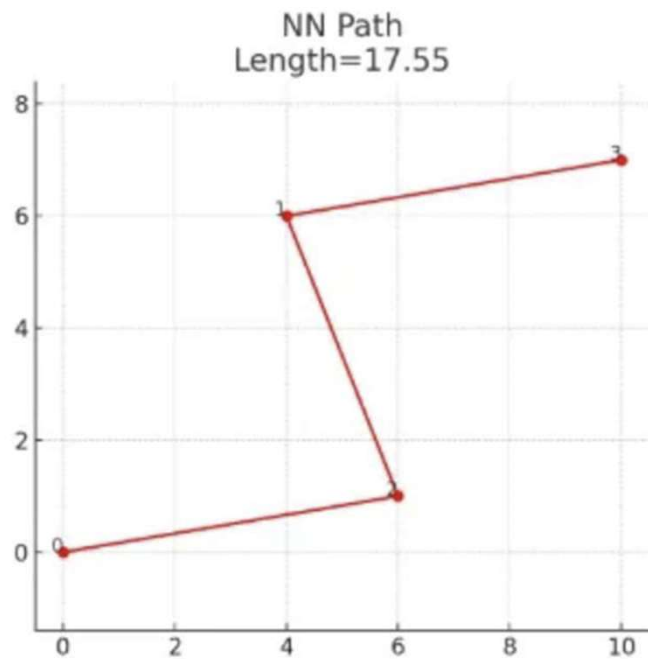


[사진 7] 고객 UI 화면(실시간 주행 영상 및 로봇 위치 확인)

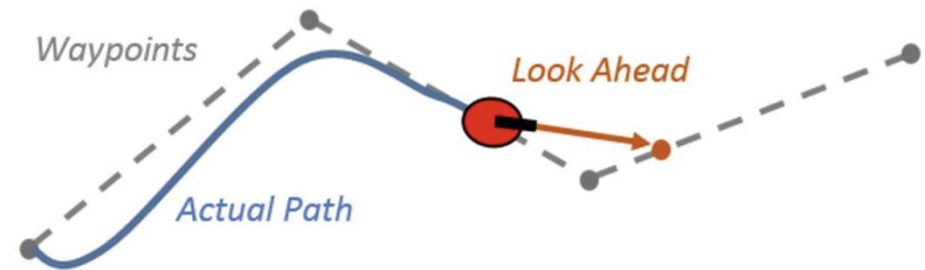
한강 자율 주행 배달 로봇

■ 경로 생성 및 추종

- 두 건 이상의 배달을 처리하기 위해 가까운 고객의 위치 순으로 경로를 생성
- 생성한 경로를 따라가기 위해 Pure-Pursuit Controller를 사용.



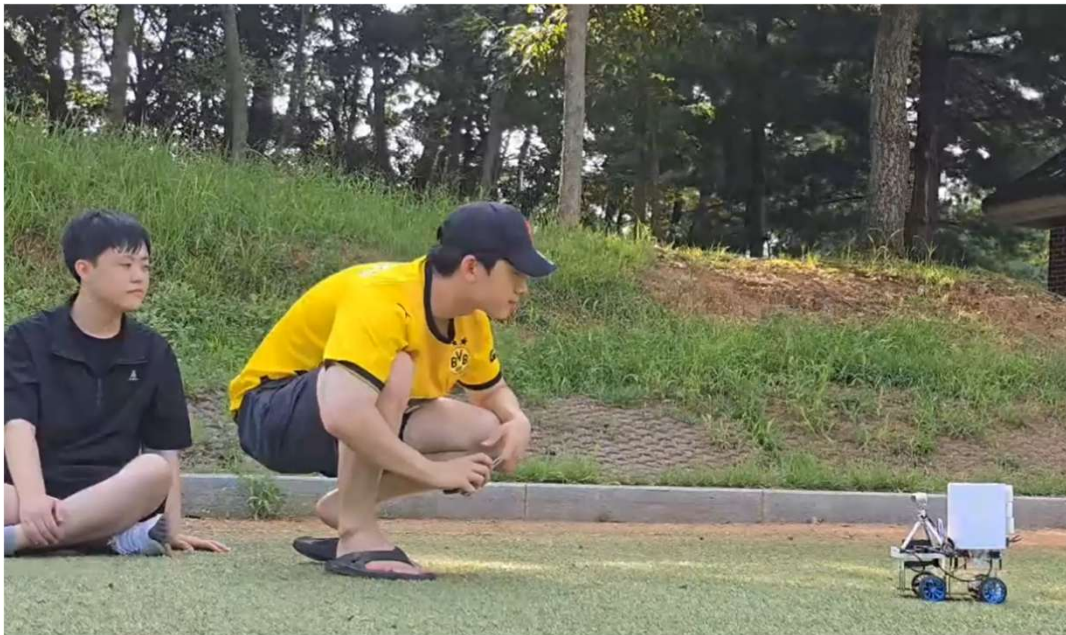
[사진 8] 최단 경로 생성 예시



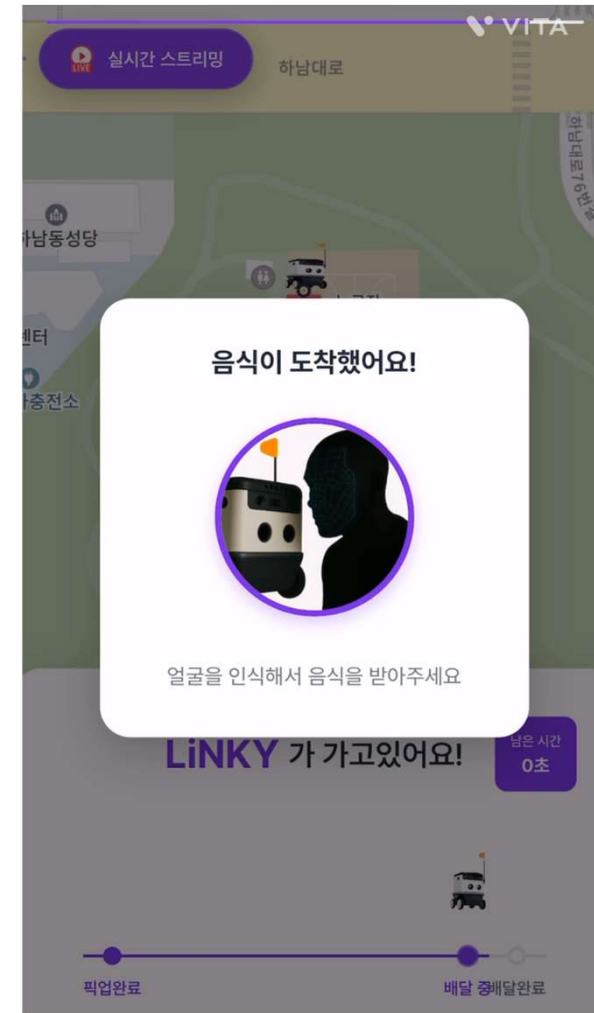
[사진 9] Pure Pursuit 예시

■ 얼굴 인식 (FaceNet)

- 사전에 입력받은 고객 사진과 실시간 입력 이미지를 비교하여 고객 얼굴 인식.
- FaceNet 딥러닝 모델을 사용하여 얼굴 이미지 비교 수행.



[사진 10] LiNKY 얼굴 인식



[사진 11] LiNKY 얼굴 인식 (사용자 UI)

한강 자율 주행 배달 로봇

■ 아쉬운 점

- 1) 초기 설계에서 MQTT 대신 gRPC를 이용하여 로봇 – 백엔드 – 프론트엔드 통신을 구현하려 했음. 그 이유는 gRPC가 사용하는 protobuf의 직렬화 특성을 이용하면 실시간 영상을 비롯한 각종 정보들을 빠르게 주고 받을 수 있을 것이라 판단했음. 서버 – 로봇 간의 통신은 완벽했지만, 브라우저에서 gRPC를 사용하기 위해선 webrtc라는 프로토콜을 사용해야 했음. 이마저도 envoy proxy 서버를 따로 두어 서버 통신을 중계 해야 했는데, Vue의 Vite와의 호환이 맞지 않음

=> 모든 시스템을 gRPC에서 REST 및 SSE 구조로 변경.

- 1) WebRTC를 이용하여 실시간 영상 스트리밍 기능을 구현하기 위해 C++ 기반 ffmpeg 파이프라인을 설계하였음. 하지만 백엔드 서버에서 사용하는 OpenVIDU와 ffmpeg의 호환이 맞지 않았으며, WebRTC Signaling을 직접 구현할 시간이 충분하지 않았음.

=> 이미지를 base64로 변환하여 MQTT토픽에 담아 송신하는 방식으로 기능을 구현

2025.04 ~ 2025.05 (3인)

Web 인터페이스 기반 로봇팔 자동화 공정 시스템 및 디지털 트윈

■ 프로젝트 개요

Git : https://github.com/YooJuno/SSAFY-Smart_Factory

목표 : 로봇의 모션제어를 웹 인터페이스를 통해 수행

배경 : 산업 및 협동 로봇의 수요 증가와 안전 사고 예방.

주요 기능 : Dobot제어, RoboDK 실행, 영상 스트리밍, 객체 검출

■ 담당 역할 및 기여

역할 : 팀장, Project Manager.

기여 : ROS2 시스템 및 서버 개발, 영상처리, Dobot 모션 제어.

■ 사용 기술

ROS2 : 영상처리 및 Dobot 제어, 웹서버와의 통신 수행.

TCP : RoboDK <=> Server, Conveyor Belt <=> Server 간의 통신 수행.

Flask : 웹 서버 및 인터페이스 구현.

Rest API : 웹서버와 ROS2 Server 간의 이미지 및 로봇 상태 정보 통신.

정기구학 : Dobot의 End-Effector 위치 계산.

OpenCV : Perspective Transformation 및 YOLO 수행.

YOLOv5 : 컨베이어 벨트 위의 객체 검출

LLM : ChatGPT API를 활용하여 사용자의 대화식 입력을 로봇 명령어로 변환



[사진 1] Web Interface + Dobot Magician + Conveyor Belt + RealSense D435i + RoboDK

■ 구현 사항

- 카메라 입력 이미지에서 Coordinate Board 부분을 추출하여 Perspective Transformation 수행. 원근이 제거된 RoI 이미지 추출
- RoI 이미지 실시간 웹 스트리밍 및 로봇의 상태 표시
- 웹상의 RoI 이미지에서 목적지 위치를 클릭하여 로봇 제어.
- Slide Bar로 로봇을 세밀하게 제어.
- 채팅창에 명령을 대화식으로 입력하여 로봇 제어.
- 컨베이어 벨트 위에 객체가 감지되면 컨베이어 벨트를 작동 시키고, 사전에 학습한 YOLO 모델을 이용해 분류 수행.
- 동시에 RoboDK 시뮬레이션 프로그램 실행.

■ 문제 해결 사례

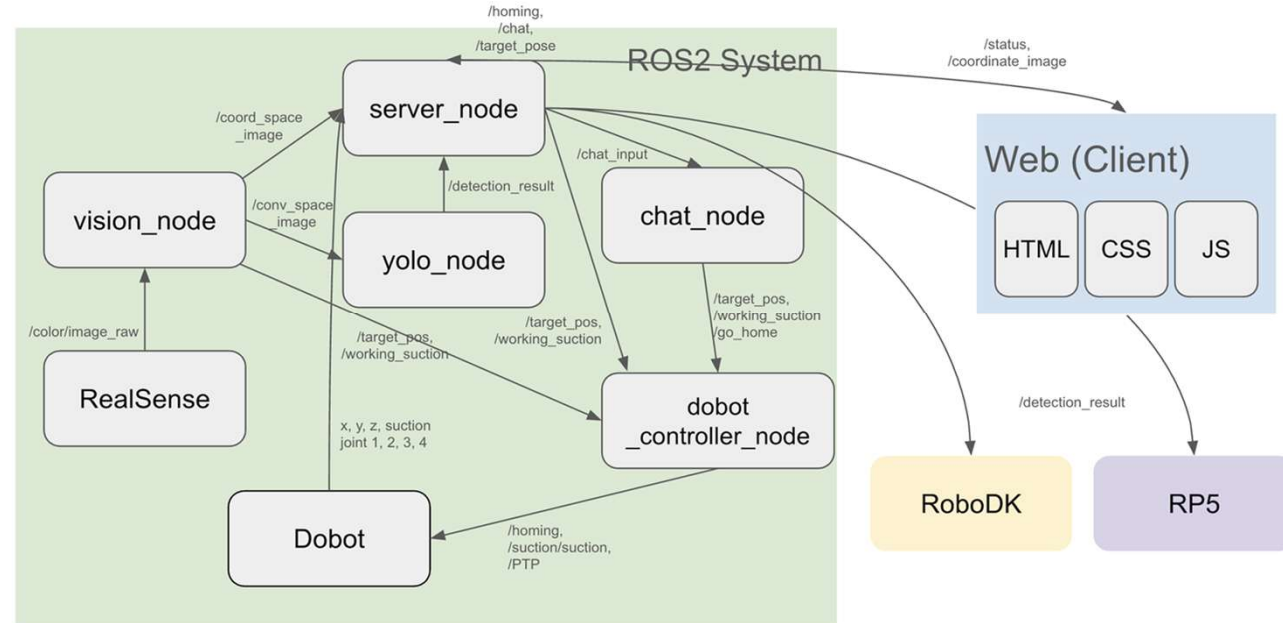
문제 : 로봇의 End-Effector 위치가 Suction-Cup이 아닌 마지막 관절의 위치로 반영이 됨.

해결 방법

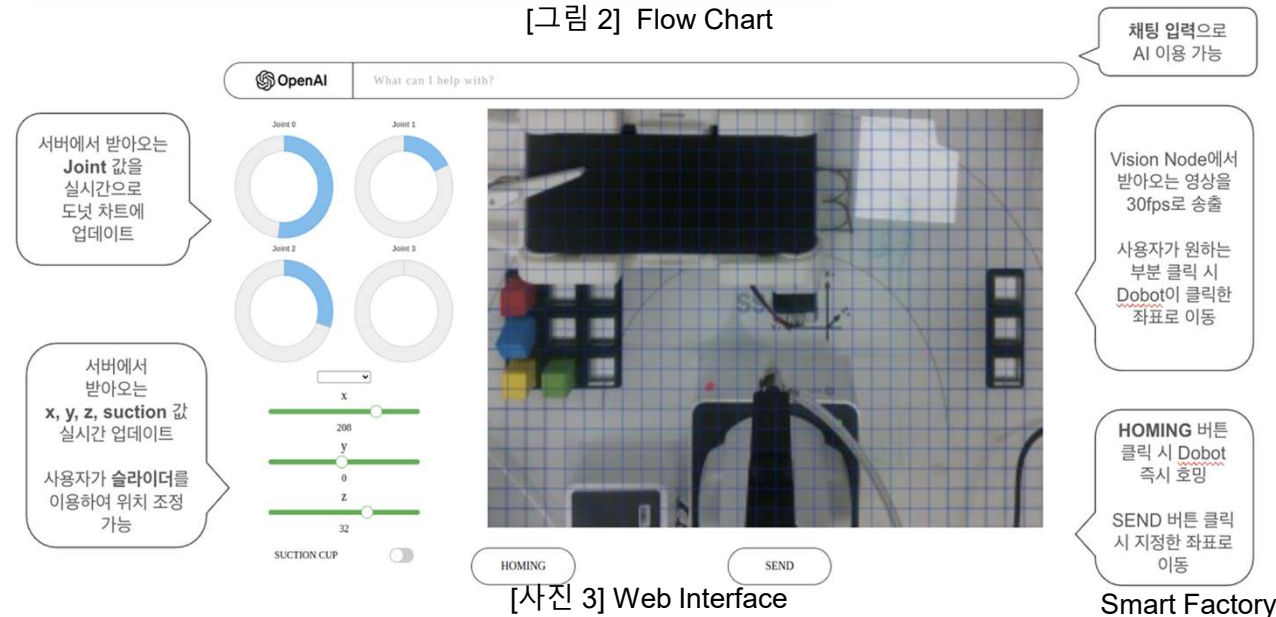
- 마지막 관절부터 Suction-Cup까지의 거리를 직접 측정.
- Forward Kinematics를 이용하여 End-Effector의 위치를 직접 계산

■ 프로젝트 회고

- 실시간 영상 스트리밍의 속도 개선을 위해 RTP 사용 필요.
- Perspective Transformation으로 원근은 제거했지만 지면으로부터 솟아 있는 객체의 경우 옆보습이 보이게 됨.



[그림 2] Flow Chart

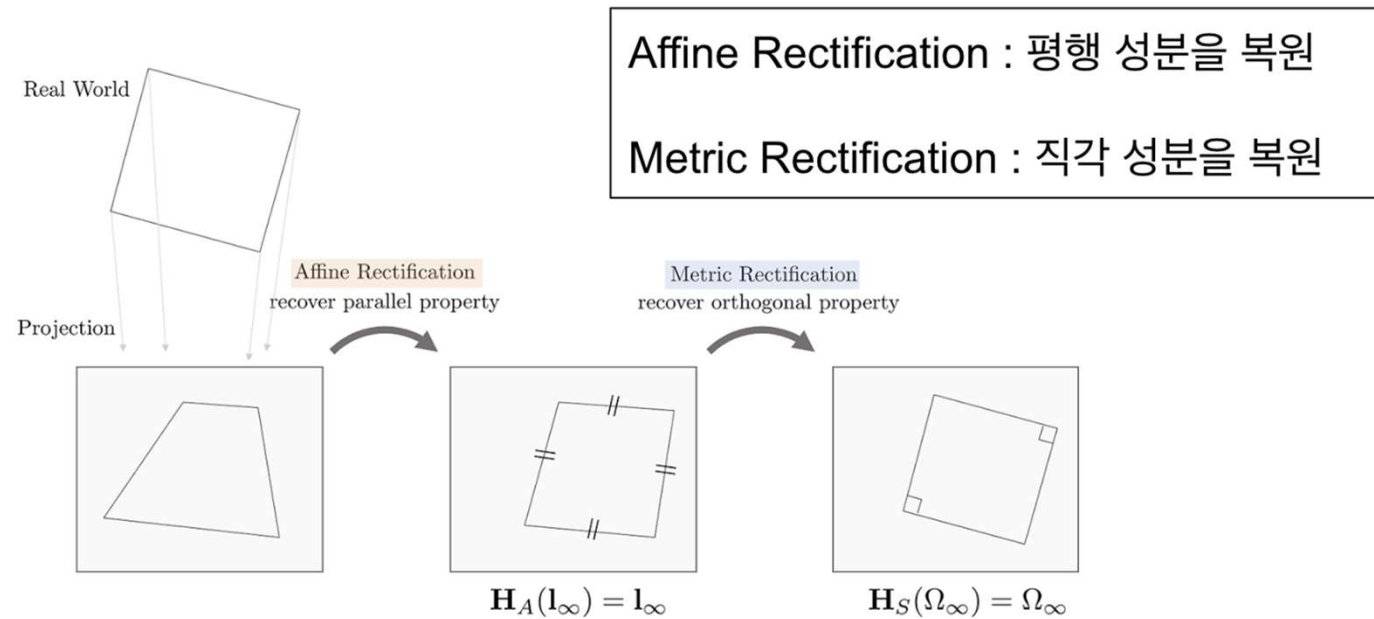


[사진 3] Web Interface

Smart Factory

■ Perspective Transformation

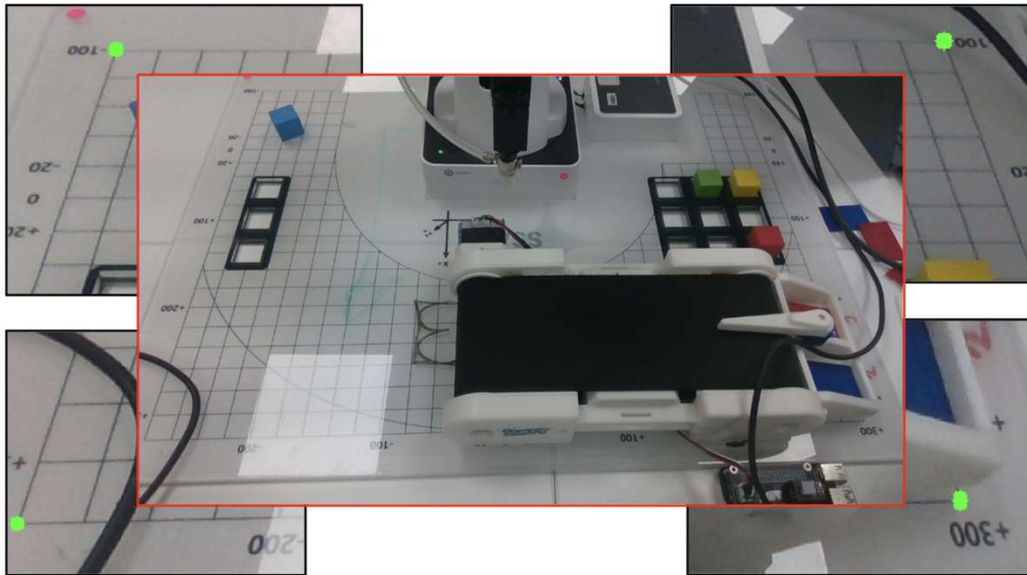
- 실제 세계에서는 직사각형이지만 원근으로 인해 왜곡이 생긴 이미지의 사다리꼴 부분에서 평행 성분과 직각 성분을 복원.
- 이를 위해선 Affine Rectification과 Metric Rectification을 수행해야 하지만, 특정 RoI(Region of Interest)에 대해서만 수행할 경우 Perspective Transformation 으로도 사각형 복원이 가능.



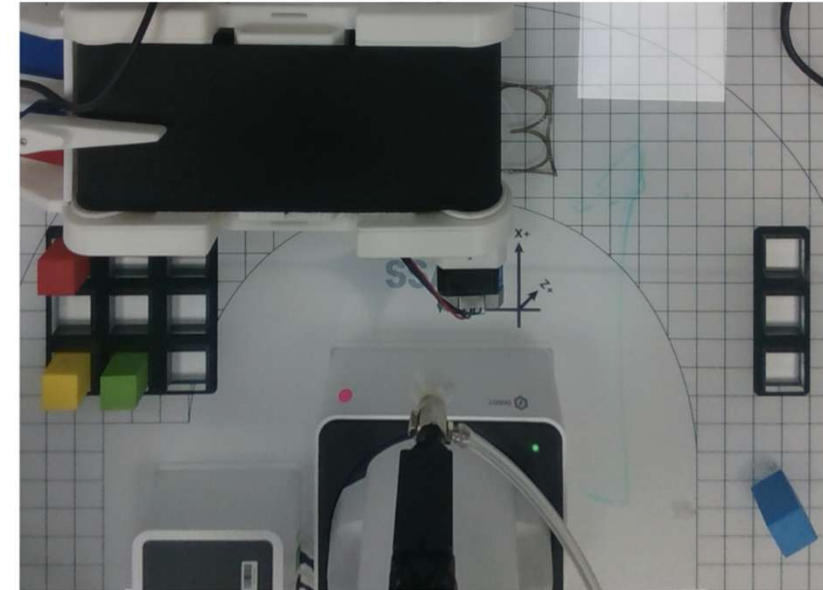
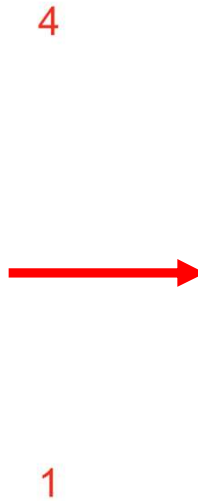
[사진 4] Affine & Metric Rectification

■ Perspective Transformation - How to?

- 실제 세계에서 보았을 때 좌상단 부터 시계방향으로 점을 직접 찍어줌.
- 원근이 제거된 이미지 추출하여 실시간으로 웹서버로 송신.
- 컨베이어 벨트 이미지만 추출하여 YOLO Node로 송신.



[사진 5] Perspective Transformation 진행 과정



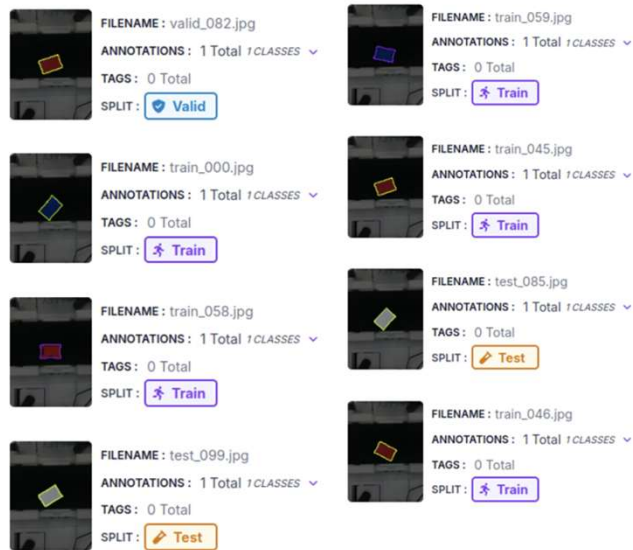
[사진 6] 원근이 제거된 Coordinate Board



[사진 7] 원근이 제거된 Conveyor Belt

■ YOLO 객체 검출

- 객체를 촬영하여 이미지 획득.
- Roboflow 를 이용하여 데이터 이미지 라벨링 및 전처리 수행.
- Chat GPT을 이용하여 증강 데이터 생성.



[사진 8] Roboflow를 이용한 데이터 라벨링

여기 시각화된 예시 5장을 확인해보세요.

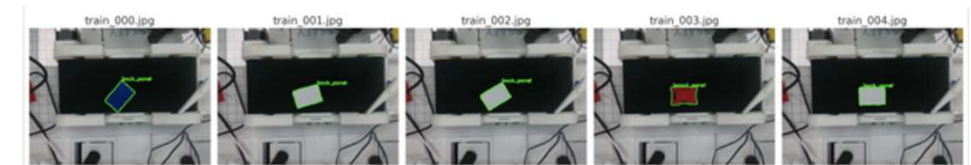
각 이미지에 대해 초록색 다각형은 실제로 검출된 ****객체의 외곽선(Polygon)****이며, 클래스 라벨 (`back_panel` , `board_panel`)도 함께 표시됩니다.

보시다시피:

- 각 패널은 벨트 중앙에 잘 배치되어 있고
- 외곽선을 따라 라벨링되어 있습니다
- 패널이 회전한 경우도 정확히 라벨링됩니다

추가적으로 확인하고 싶은 이미지가 있다면 알려주세요.

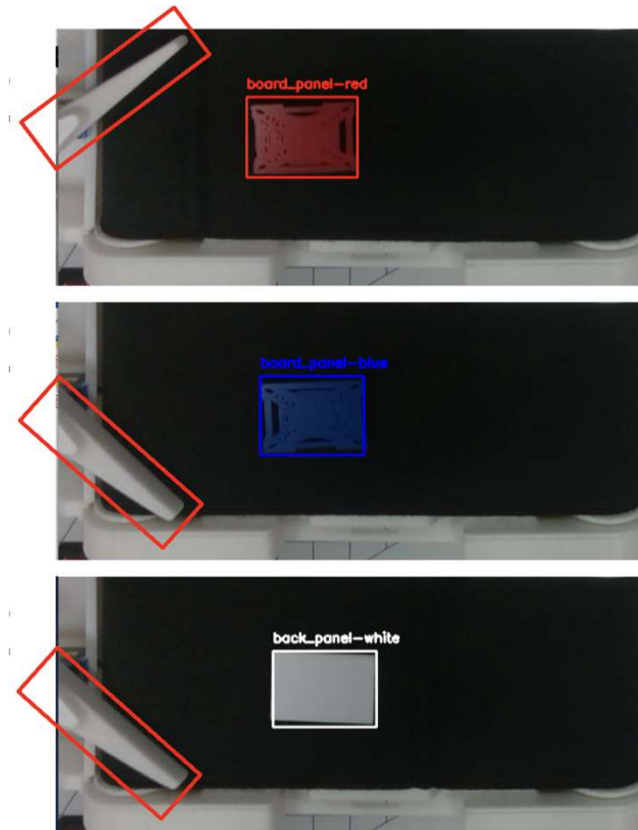
또는 다른 형식(COCO JSON, Pascal VOC XML 등)으로 변환이 필요하다면 그것도 가능합니다. [\[↗\]](#)



[사진 9] Chat GPT를 활용한 증강 이미지 생성

■ YOLO 객체 검출

- 빨간색 객체를 불량품으로 판단하여 분류를 진행.



[사진 10] 객체 검출 후 분류

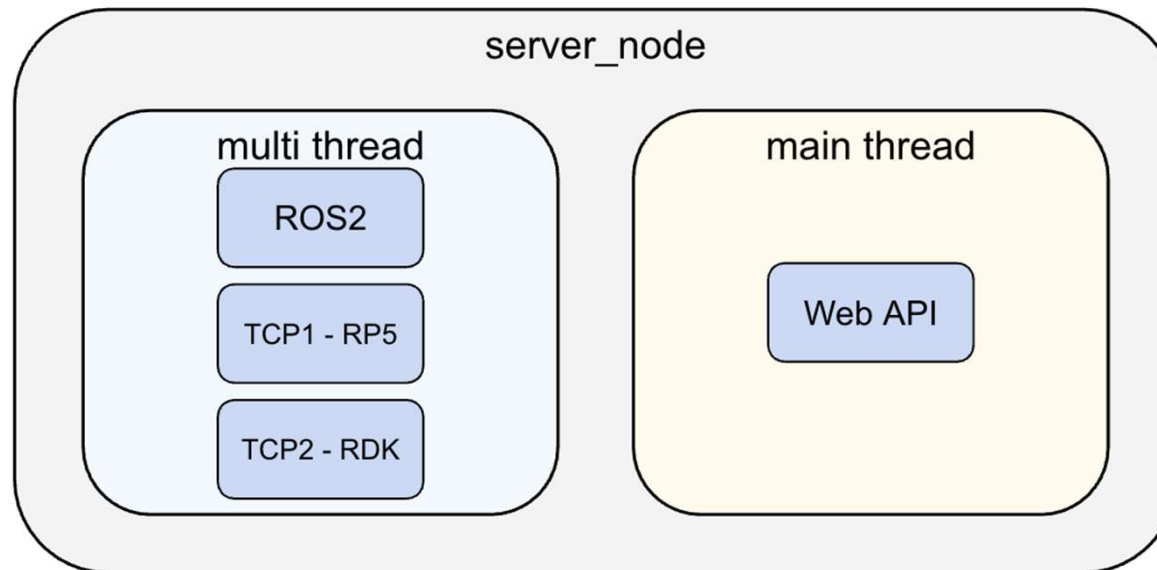
■ SERVER

메인 쓰레드 외에 3개의 멀티 쓰레드를 사용하여 실시간 비동기 통신을 수행하였음.

- 1) **ROS2** : Dobot과 영상처리 노드와의 데이터 통신을 담당.
- 2) **TCP1** : Conveyor Belt를 구동하는 라즈베리파이5와의 TCP 통신을 담당.
- 3) **TCP2** : RoboDK를 실행중인 Window PC와의 TCP 통신을 담당
- 4) **Web API** : 웹 클라이언트로 부터의 접근을 대기.

■ 멀티 쓰레드를 사용한 이유?

클라이언트의 수가 동적으로 변하지 않고 그 수가 컴퓨터 리소스의 성능에 영향을 주기엔 미비하다 판단하여 멀티 쓰레드를 사용함으로써 더욱 안정적인 시스템을 구현하였음.



[사진 10] 멀티 쓰레드 구조도

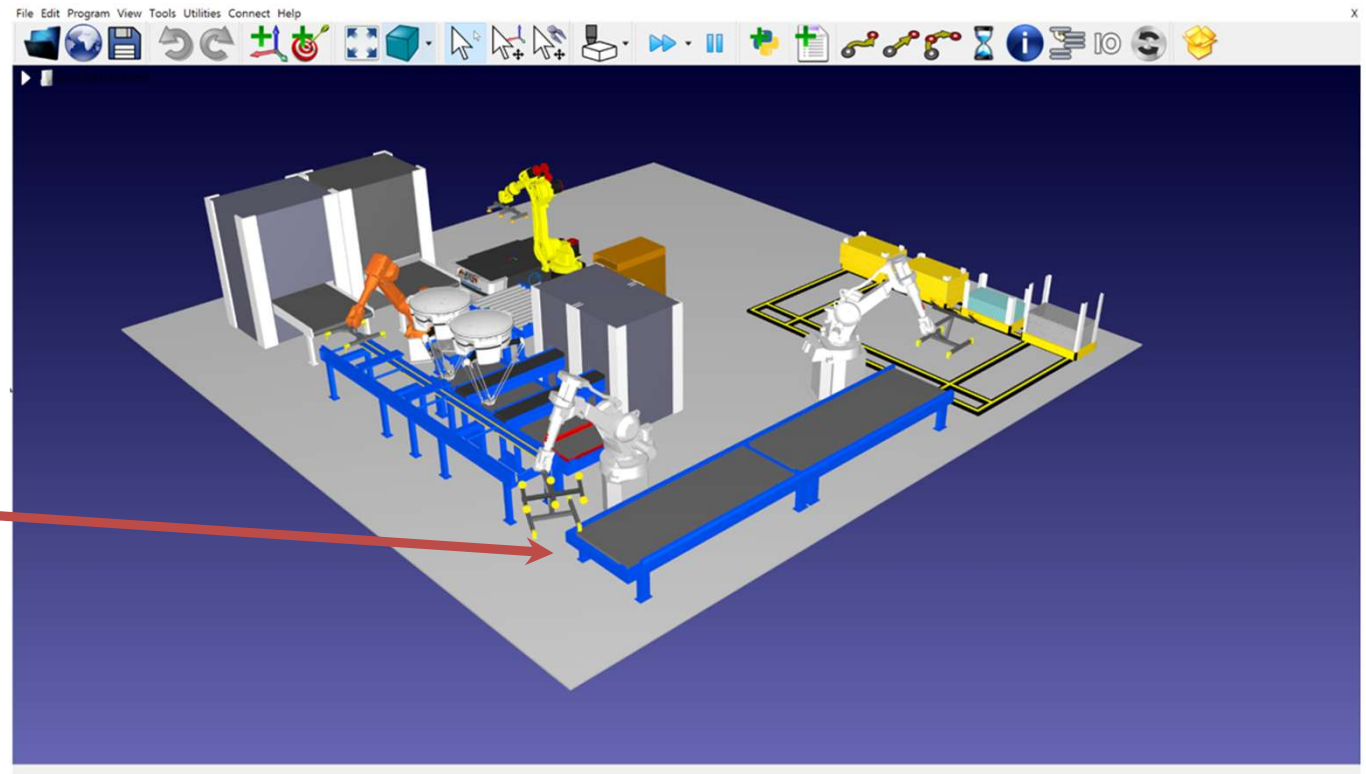
■ RoboDK

컨베이어 벨트에서 객체를 인식하여 검출한 이후의 시나리오에 대해 RoboDK로 구현함.

- 1) 컨베이어 벨트 위 객체 인식 후 분류
- 2) 검출 결과를 RoboDK 서버로 송신(TCP)
- 3) 모든 객체의 Way Point를 직접 설정하여 디스플레이 제작 공정 시나리오를 기획.
- 4) RoboDK 설계를 위해 공정 과정에 대한 이해와 기구학(FK, IK) 학습.



[사진 11] 컨베이어 벨트 실물



[사진 12] RoboDK 공정 시뮬레이션

■ Demo Video



[동영상] <https://www.youtube.com/watch?v=G8R1HWxcDtk>

2025.02 ~ 2025.03 (1인)

로봇 센서 데이터와 영상의 실시간 분석 및 웹 시각화 시스템

■ 프로젝트 개요

Git : <https://github.com/YooJuno/SSAFY-Web>

목표 : Robot 영상을 서버에서 영상처리를 수행. 그 결과를 웹 페이지에 송출.

배경 : 서버 분산 처리 수요 증가.

주요 기능 : 서버 분산 처리, Robot 웹 인터페이스, 영상 내 객체 검출.

■ 담당 역할 및 기여

역할 : Full Stack Developer.

기여 : 웹 인터페이스 구현, Server 통신 시스템 개발, 영상처리.

■ 사용 기술

Vue.JS : 비교적 간단한 구현 방식과 효율적인 통신 구현 가능.

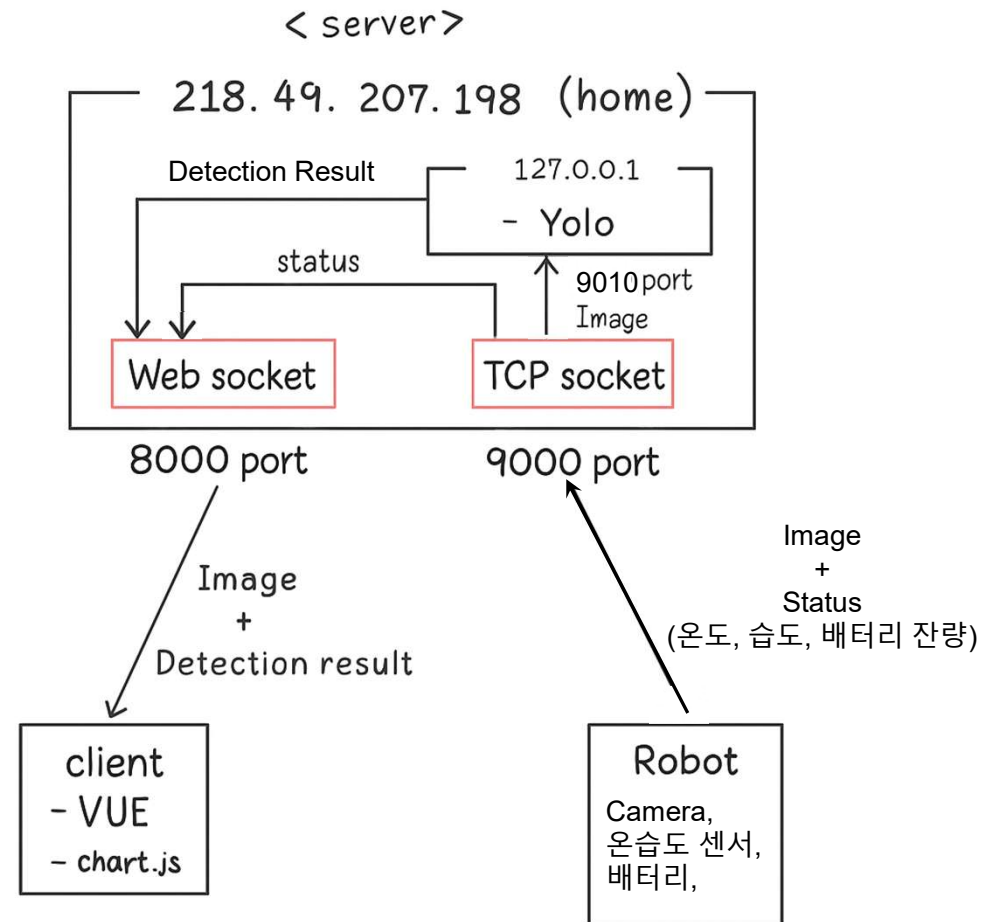
Node.JS : 웹서버 - CCTV - DB - 분산처리 노드 간의 통신을 위한 중계 역할.

MySQL : Log 저장.

Nginx : Web page 배포

TCP/Web Socket : 영상 데이터 통신.

YOLOv9 : 영상 내 객체 검출.



[사진 1] System Architecture

■ 구현 사항

[Front-End]

- 센서 데이터를 실시간으로 수신하여 Chart로 표시
- 객체 인식 결과와 이미지를 실시간으로 수신하여 Chart 표시 및 영상 스트리밍

[Back-end]

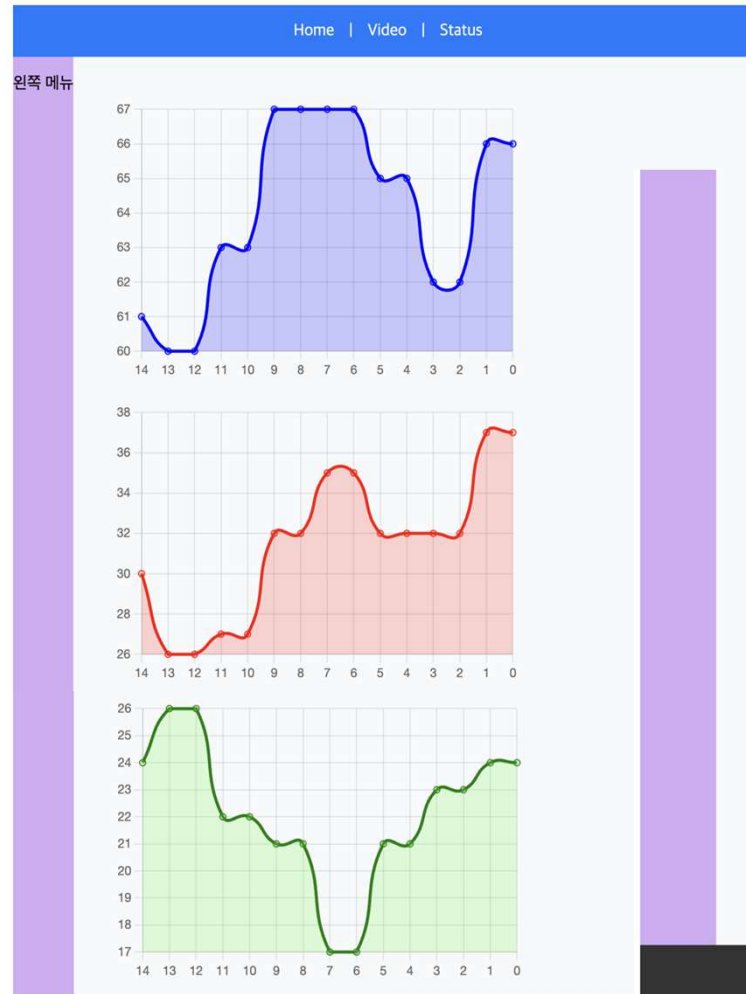
- 서버에서는 Robot(Raspberry Pi) 영상을 수신하여 Localhost에서 동작중인 YOLO 서버로 이미지를 송신.
- YOLO 서버에서는 이미지를 수신하여 그 결과를 서버로 회신.
- 서버에서는 YOLO 결과 데이터를 수신하여 웹 서버로 송신.
- 모든 과정을 비동기적으로 처리하여 통신 성능을 높였음.

[Robot]

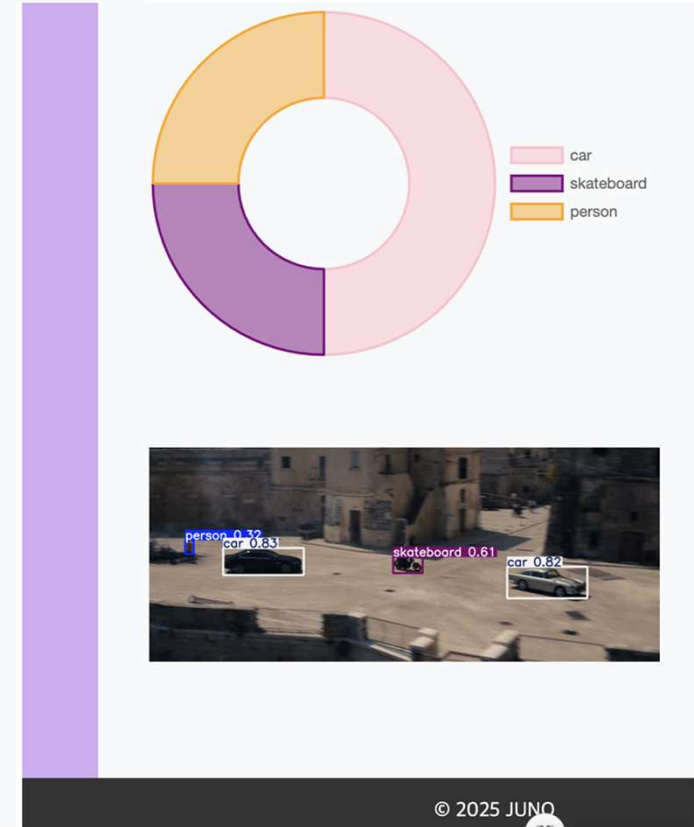
- 실시간 센서 데이터 및 영상을 서버로 송신
- 라즈베리 파이 카메라 고장으로 영상을 실시간 촬영 이미지가 아닌 별도의 동영상으로 대체하였음

■ 프로젝트 회고

- 실시간 영상 스트리밍의 속도 개선을 위해 RTP 사용 필요.
- 서버에서 로드 밸런싱을 적용하여 분산 처리 기능의 확장이 가능 할 것으로 보임.
- Log 저장 기능은 구현했지만 그 쓰임이 애매해 데모에서는 활용하지 않았음. DB 기능을 활용하는 방향으로 보완 필요.



[사진 2] 실시간 센서 데이터 차트



[사진 3] YOLO 검출 결과

2024.07 ~ 2024.08 (1인)

블랙박스 영상을 활용한 차량의 속도 추정

■ 프로젝트 개요

Git : 보안상 공개 불가

회사 : GMD SOFT(인턴)

목표 : 블랙박스 이미지를 이용하여 차량의 속도를 추정.

배경 : 차량 사고시에 단속 카메라가 없는 경우 과속 여부를 판별하기 어려움.

주요 기능 : 해당 블랙박스 영상 시점의 차량의 속도를 계산.

■ 담당 역할 및 기여

역할 : 1인 개발.

기여 : 3D Reconstruction, Image Preprocessing, Real-to-Pixel 스케일 변환, 차량 속도 계산.

■ 사용 기술

SfM (COLMAP) : 블랙박스 영상을 활용하여 3D Reconstruction 수행.

OpenCV : 블랙 박스 영상 Masking 및 전처리 수행.

Real-To-Pixel : 실제 세계와 재구성한 3D 세계와의 스케일을 계산

■ 구현 사항

- 블랙 박스 영상 전처리(Masking).
- COLMAP 을 사용하여 블랙박스 주행영상을 3D 재구성.
- 실제 세계와 3D 세계와의 스케일 값을 계산 및 속도 추정.



[사진 1] 블랙박스 영상을 활용한 속도 추정 예시

■ 문제 해결 사례

문제 : 3D 재구성이 불규칙 적으로 이루어지지 않음을 발견.

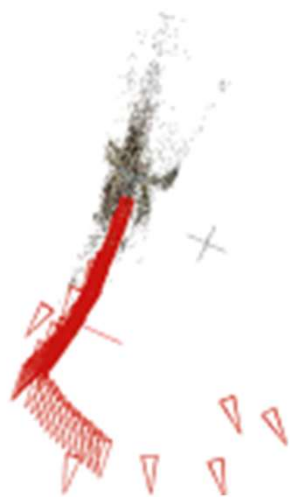
해결 방법 : 입력 이미지에서 차량의 보닛 이미지(노이즈) 를 마스킹하여 제거함으로써 보다 나은 Point Cloud를 얻을 수 있었음.

■ 블랙박스 영상 전처리 (Masking)

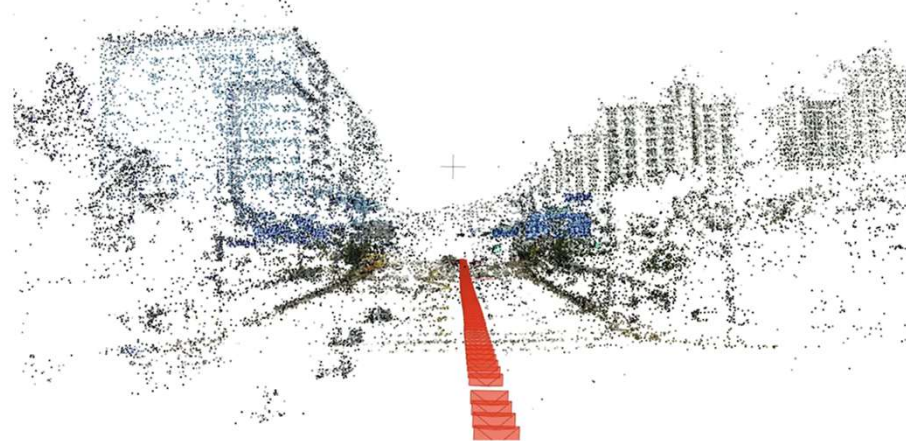
- 입력으로 들어오는 영상에는 상단에 블랙박스 정보, 하단에 차량 이미지가 포함되어 있다.
- 이 정보들은 Point Cloud를 생성함에 있어 Noise와 같기 때문에 제거해야 한다.



[사진 2] 블랙 박스 영상 Masking 과정



[사진 3] Masking 전 결과물



[사진 4] Masking 후 결과물

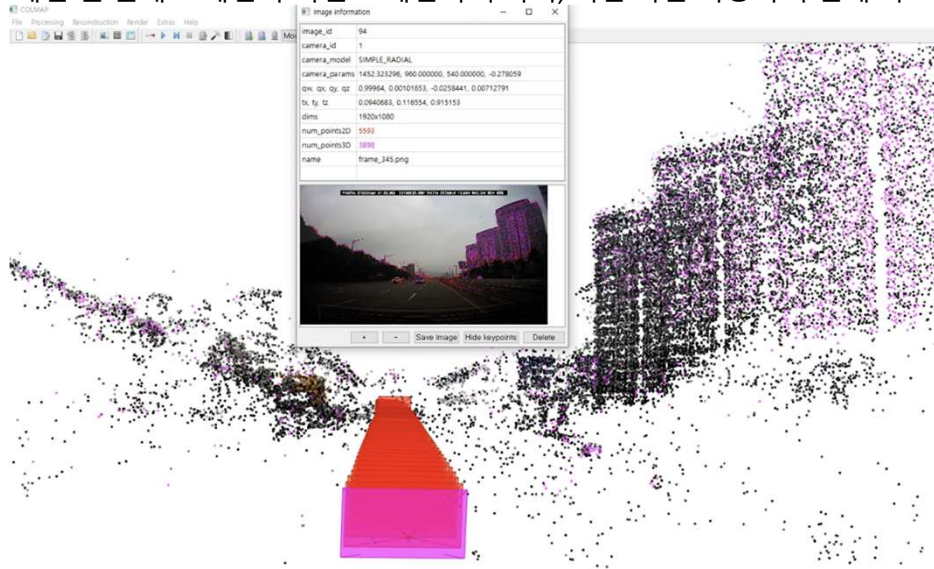
블랙박스 영상을 활용한 차량의 속도 추정

■ Real-To-Pixel Scale 구하기

- Real-To-Pixel Scale은 쉽게 말해 현실 세계에서 1m가 3D 재구성한 세계에서 몇 pixel인지를 구하는 과정이다.
- 이를 이용하여 3D 세계에서 차량의 위치 별 시간 정보를 이용하여 속도를 구해낼 수 있다.

■ How To?

- 1) 해당 구간 내의 현실세계에서 눈에 띄는 특징을 지닌 랜드마크를 선별하여 네이버 지도로 실제 길이를 측정.
(가로등 간격을 주로 사용함)
- 2) 재구성한 세계에서 동일한 랜드마크를 찾아 Pixel 길이를 측정.
- 3) 똑같은 구간에 대해 m, pixel 비율 값을 찾아낸다.
- 4) 이 비율 값과 프레임 별 현재 프레임과 이전 프레임과의 거리, 시간 차를 이용하여 실제 속도를 계산해 냄.



[사진 4] 가로등의 Pixel 위치 확인



[사진 3] 네이버 지도를 활용한 랜드마크 거리 측정

블랙박스 영상을 활용한 차량의 속도 추정

■ Demo Video

- 총 세 개의 주행 비디오를 사용했음.
- 세 개의 영상 모두 일정 구간에서는 좋은 성능을 보임(+/- 3km/h)
- 일정 구간은 대부분 랜드마크로 지정한 위치와 같거나 근처임.
- 하지만 결국 오차는 점점 커지는데, 후방 카메라의 경우 그 오차가 250km/h만큼이나 벌어짐.



[후방 카메라 1]

https://www.youtube.com/watch?v=_HwGZA1f8t0



[전방 카메라 1]

<https://www.youtube.com/watch?v=cIEgDyLUE0I&t=13s>



[후방 카메라 2]

<https://www.youtube.com/watch?v=oEf1EfqPtWA>

블랙박스 영상을 활용한 차량의 속도 추정

■ 왜 오차가 생기는가?

- COLMAP은 아주 훌륭한 3D Reconstruction 프로그램이다.
- 훌륭하다고 말할 수 있는 이유는 객체를 재구성 할 때 그 카메라 종류에 덜 국한되며 유연성과 정확도로 객체를 사실적으로 표현하기 때문이다.
- 하지만 직선으로 주행하는 블랙박스 영상의 특성은 COLMAP에서 오차를 유발한다.
- 사실적인 묘사는 가능하지만 재구성한 세계의 스케일은 카메라가 이동하면서 달라진다.
- 이는 Real-To-Pixel 변환 과정에서 오차를 유발하게 된다 이는 랜드마크 주변에서는 속도가 비교적 정확하지만 멀어질 수록 오차가 커지는 이유로 보인다.

■ 프로젝트 회고

- 특정 구간에 대해 납득 가능할 만한 오차($\pm 3\text{km/h}$)를 보이기도 하지만 크게는 250km/h 까지의 오차도 보인다.
- 납득 가능할 만한 오차가 나오는 구간은 스케일 비율을 구하기 위해 레퍼런스 길이를 측정했던 구간에서 보였으며, 이 곳에서 멀어질 수록 속도의 오차가 점점 커졌다.
- 3D재구성을 어떻게 수행하는지에 따라 결과가 크게 바뀐다.
- COLMAP에 대해 좀 더 공부하여 더욱 사실적으로 3D 재구성을 하거나 COLMAP 외의 다른 프로그램을 사용한다면 더욱 정밀한 속도 측정이 가능할 수도 있음.

2024.04 ~ 2024.06 (5인)

CCTV 영상 내 객체 속도 추정

■ 프로젝트 개요

Git : 보안상 공개 불가

발주처 : GMD SOFT (산학 과제)

목표 : 경쟁사 제품 조사 및 프로그램 개발

배경 : 해마다 증가하는 차량 사고. 그 중 상당 수가 과속을 입증하기 힘들.

주요 기능 : CCTV 영상에 찍힌 차량의 주행 속도 추정.

■ 담당 역할 및 기여

역할 : 선행 연구 조사 및 프로그램 개발. Demo Data 수집

기여 : 경쟁사 제품 알고리즘 파악, Perspective Transformation을 활용한 속도 추정 알고리즘 개발.

■ 사용 기술

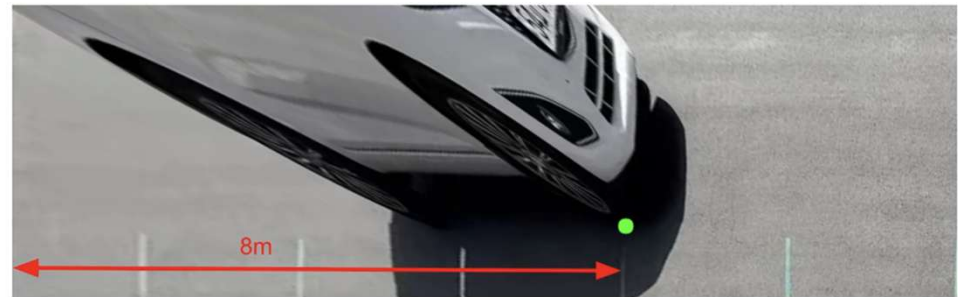
OpenCV : 영상 데이터를 다루기 위해 사용함.

Perspective Transformation : Affine & Metric Rectification 과정에서 에러가 많아 보다 단순하고 직관적인 Perspective Transformation 적용.

13km/h 속도 기존 dataset

- 실제 길이 : 8.0m, 추정 길이: 7.8m → 오차: -0.2m
- 실제 속도: 13 km/h, 추정 속도: 12.4 km/h → 오차: -0.6 km/h

length : 7.8 [m]
velocity : 12.4 [km/h]

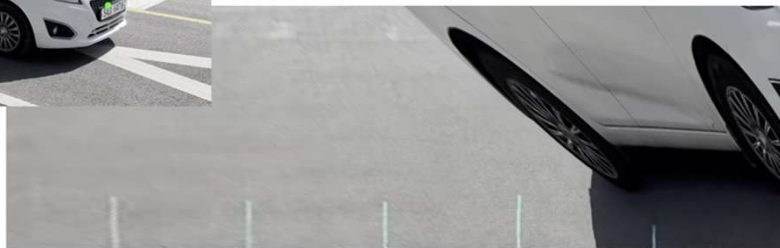


[사진 1] 차량의 속도 추정

■ 구현 사항

- CCTV 영상 내에서 차량이 주행한 지면의 직사각형 길이를 실제로 직접 측정.
- 해당 구간을 직접 주행하여 각 속도 별 데이터 셋 촬영.
- Perspective Transformation을 통해 원근이 제거된 직사각형 영역을 CCTV 영상으로부터 추출.
- 차량이 해당 영역에 들어왔을 때, 사전에 그어 놓은 초록 선을 클릭하여 해당 시점의 차량의 위치 정보를 저장.
- 차량의 현재 시간 및 위치 정보와 이전 시점과의 차이를 이용하여 속도 추정.

■ Perspective Transformation



[사진 3] Perspective Transformation을 활용하여 주행 구간의 원근을 제거한 모습

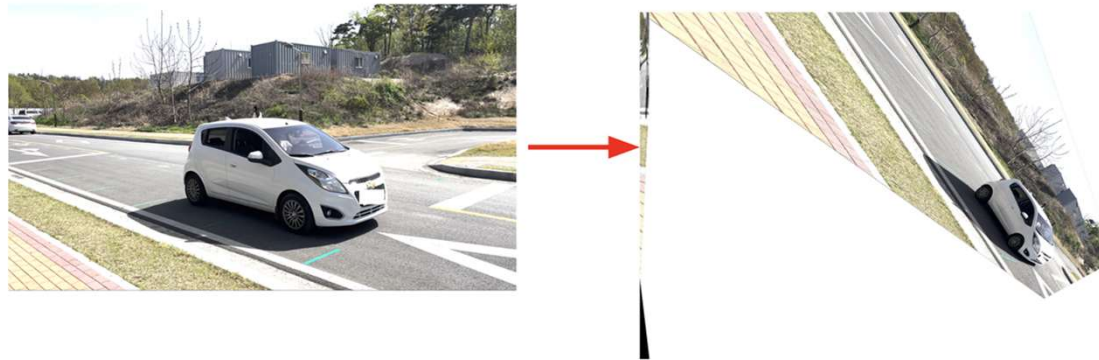


[사진 2] Dataset 촬영
(청테이프로 거리 구간을 설정했다)

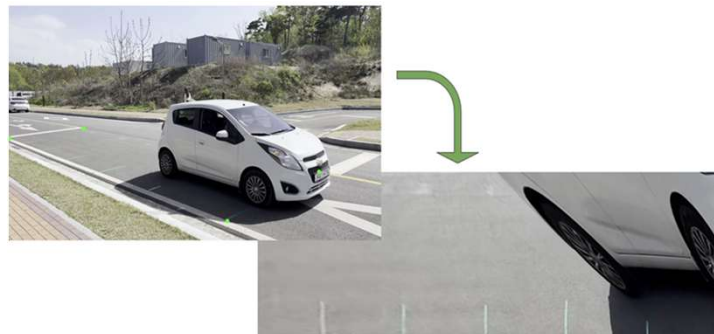
■ 문제 해결 사례

문제 : 원근을 없애기 위한 Affine & Metric Rectification을 수행함에 있어 연산량이 많고 그 결과가 직각으로 제대로 나오지 않았음

해결 방법 : 정해진 기한을 지키기 위해 다른 방법을 모색하여 Perspective Transformation을 적용.



[사진 4] Affine & Metric Rectification 오류



[사진 5] Perspective Transformation을 활용하여
주행 구간의 원근을 제거한 모습

CCTV 영상 내 객체 속도 추정

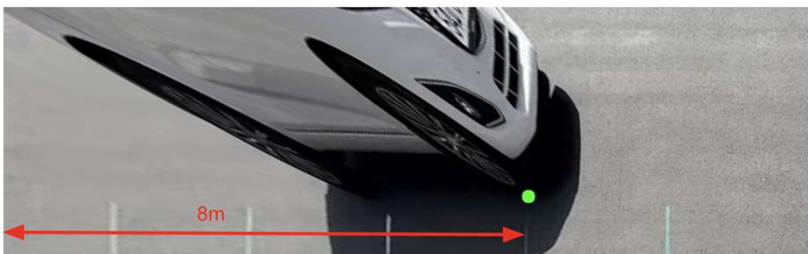
■ 결과

- 주어진 데이터셋에서 평균 0.5km/h의 오차를 보임.

13km/h 속도 dataset

- 실제 길이 : 8.0m, 추정 길이: 7.8m → 오차: -0.2m
- 실제 속도: 13 km/h, 추정 속도: 12.4 km/h → 오차: -0.6 km/h

```
length : 7.8 [m]  
velocity : 12.4 [km/h]
```

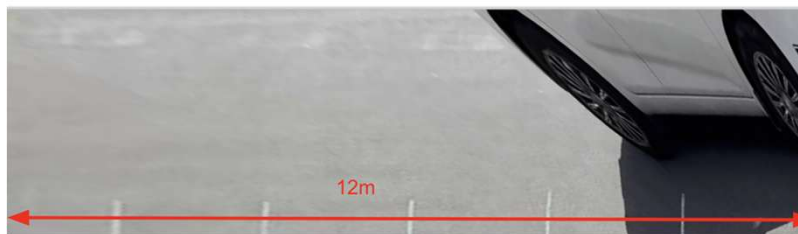


[사진 6] 13km/h dataset

16km/h 속도 dataset - 1

- 실제 길이 : 12.0 m, 추정 길이: 12.0 m → 오차: 0 m
- 실제 속도: 16 km/h, 추정 속도: 16.1 km/h → 오차: + 0.1 km/h

```
./transformed_images/8.10.jpg  
length : 12.0m  
velocity : 16.1km/h
```

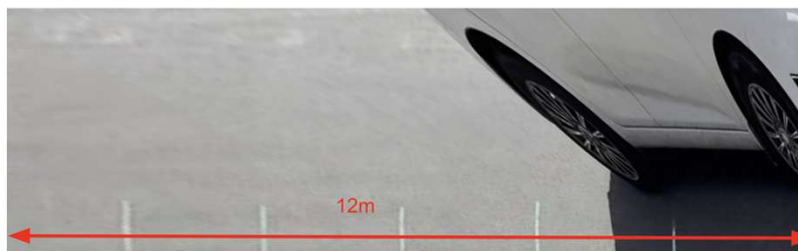


[사진 7] 16km/h dataset

18km/h 속도 dataset - 2

- 실제 길이 : 12.0 m, 추정 길이: 12.0 m → 오차: 0 m
- 실제 속도: 18 km/h, 추정 속도: 17.9 km/h → 오차: -0.1 km/h

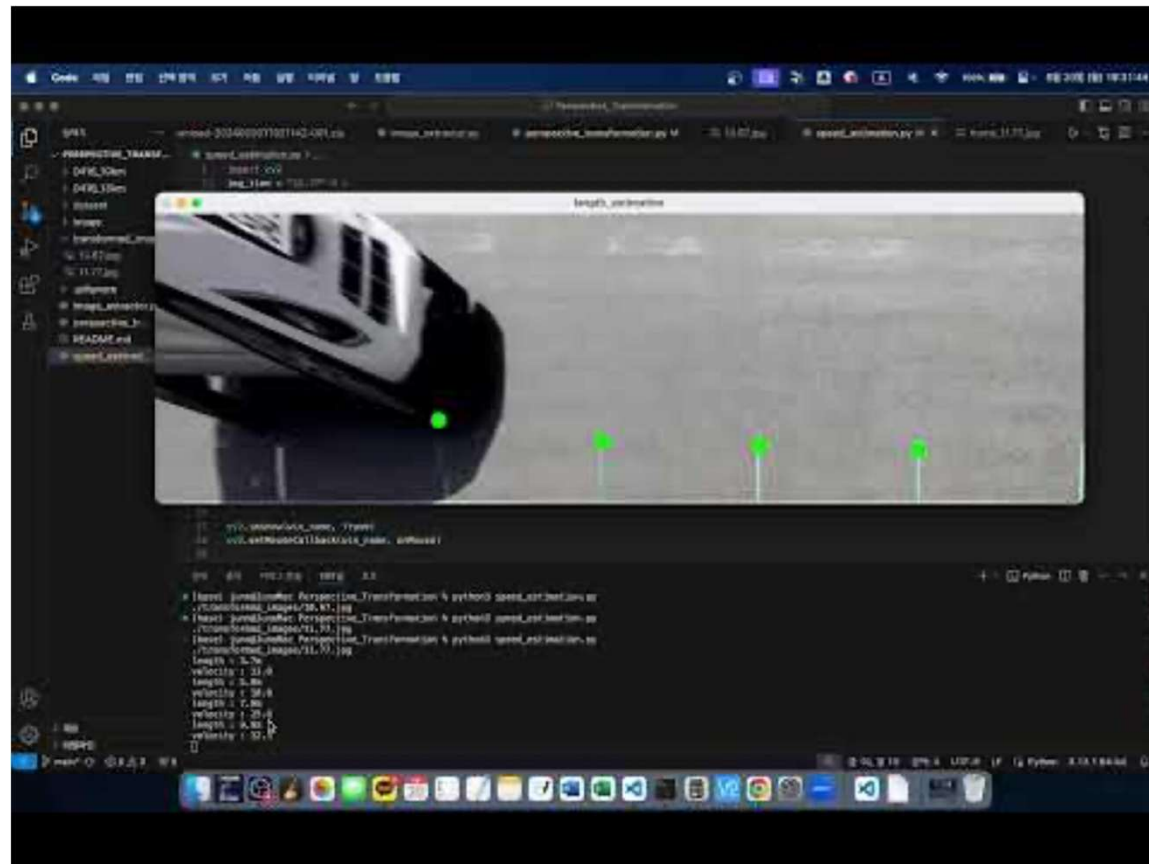
```
./transformed_images/6.00.jpg  
length : 12.0m  
velocity : 17.9km/h
```



[사진 8] 18km/h dataset

■ Demo Video

Perspective Transformation 과정과 그 결과를 세미나 형식으로 다루었습니다.



[동영상] https://www.youtube.com/watch?v=v-8i_FkTkiU

CCTV 영상 내 객체 속도 추정

■ 프로젝트 회고

- 촬영 여건 상 고속 데이터셋을 촬영하지 못해 고속 주행 환경에서의 결과물을 보장할 수 없음.
- 현재에는 Perspective Transformation을 수행하여 Roi 영역에 대해서만 원근을 제거했지만, 실제 상용화를 위해선 해당 구간 이외의 영역에 대해서도 원근이 제거된 이미지가 필요하기 때문에 Affine & Metric Rectification이 필요.
- 하지만 원근을 제거한 영상에 대해서 속도 측정이 가능함을 입증했다는 것에서 가치가 있음.

■ 성과

- 산학 과제 계약 연장
- 방학 동안 인턴쉽 기회를 얻을 수 있었음.

2024.01 ~ 2024.04 (5인)

CCTV 영상 내 객체 높이 추정

■ 프로젝트 개요

Git : 보안상 공개 불가

발주처 : GMD SOFT (산학 과제)

목표 : CCTV 영상에서 용의자의 키를 추정.

배경 : 범죄현장에서 용의자의 인상착의를 특정하는데에 어려움이 큼.

주요 기능 : Vanishing Point & Line 생성. 객체 높이 계산.

■ 담당 역할 및 기여

역할 : 선행 연구 조사 및 프로그램 개발

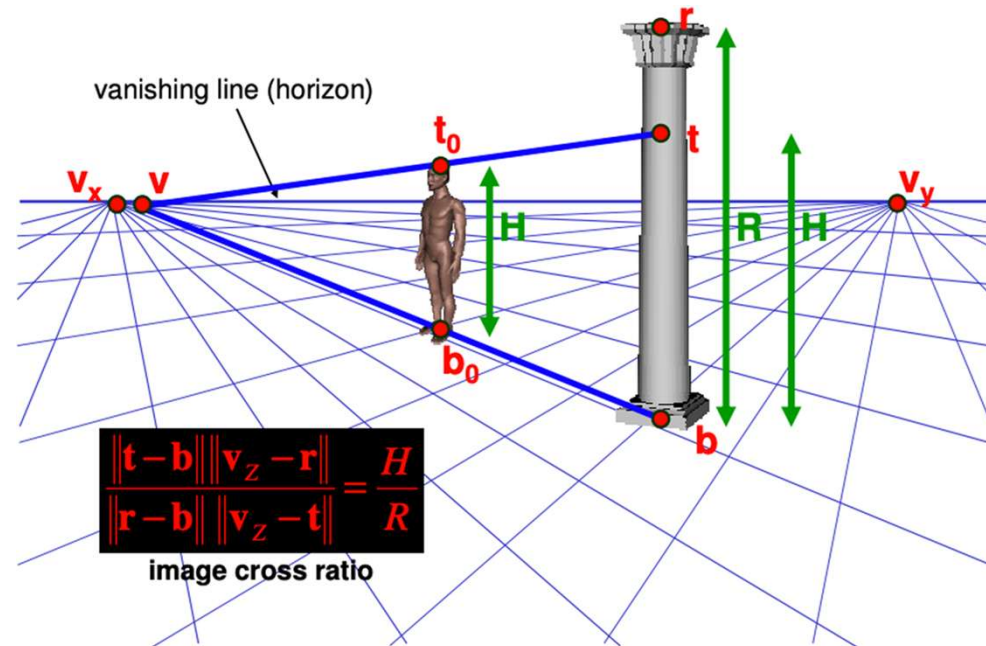
기여 : Single View Geometry(SVG) 기반 높이 추정 알고리즘 개발.

■ 사용 기술

SVG : 단안 렌즈의 원근을 이용한 객체의 높이 추정 가능

OpenCV : CCTV 영상 이미지 처리.

PyQT : 사용자의 입력을 GUI로 입력 받아 Vanishing Point 및 Reference Line 특정.

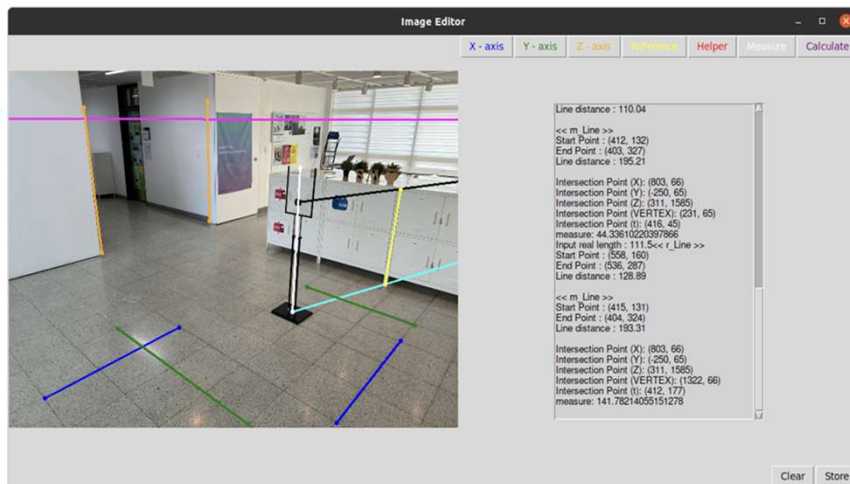


[사진 1] Single View Metrology _ Vanishing Point

■ 구현 사항

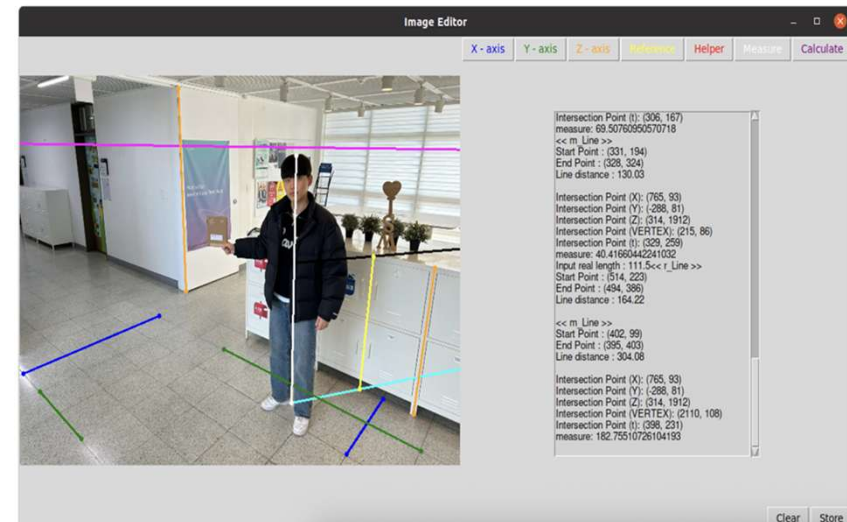
- PyQT를 통해 X, Y, Z 방향 별 평행선을 각 두 개 씩 입력(선 하나는 점 두 개를 의미)
- Reference Line과 Object Line에 해당하는 선분 또한 입력.
- Reference Line은 실제 길이 값도 함께 입력.
- X, Y, Z 각 방향 별 Vanishing Point를 생성하고, X, Y 방향의 Vanishing Point를 이용해 수평선에 해당하는 Vanishing Line 생성
- Vanishing Line과 Reference Line, Z Vanishing Point를 이용하여 비율 계산을 통해 Object의 길이를 계산해냄.

실제 Demo에서는 사물함의 높이를 이미 알고 있는 Reference Line(노란색)으로 산정하고 표지판과 사람의 키(하얀색)를 측정하였음



[사진 2] 표지판 높이 추정

오차 : + 1.7cm



오차 : + 1cm

[사진 3] 사람 높이 추정

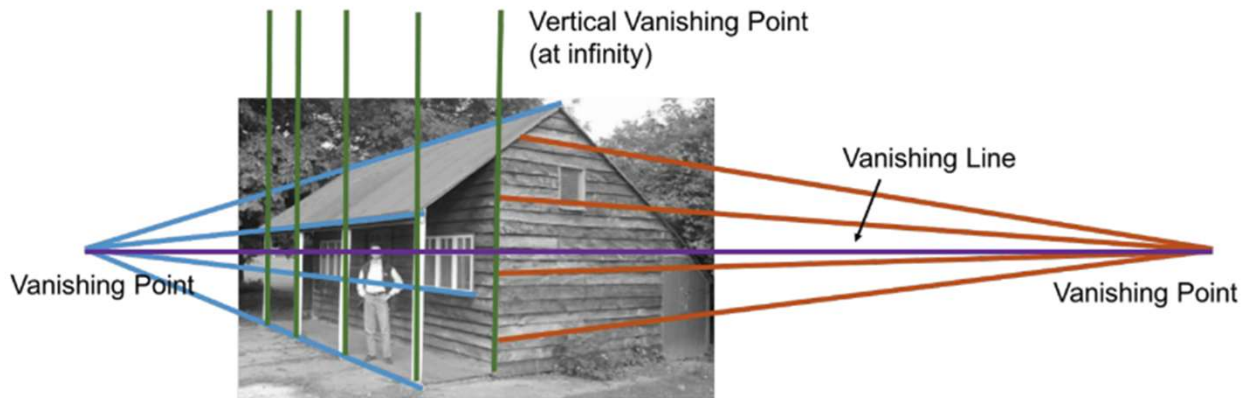
CCTV 영상 내 객체 높이 추정

■ Vanishing Point?

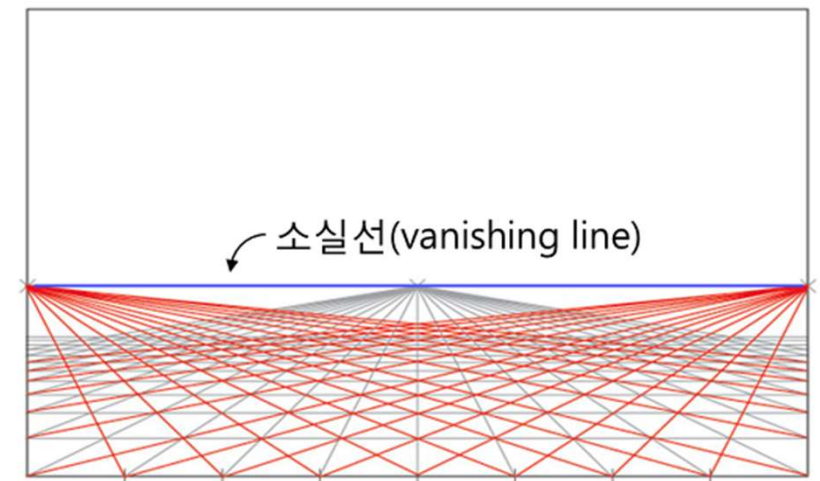
- 실제 세계에선 평행하나 이미지 상에서는 원근으로 인해 한 점에서 수렴하는 점을 의미.

■ Vanishing Line?

- 수평선과 같은 개념으로 동일한 평면상에서 각 방향으로의 Vanishing Point 들을 연결한 직선을 의미.



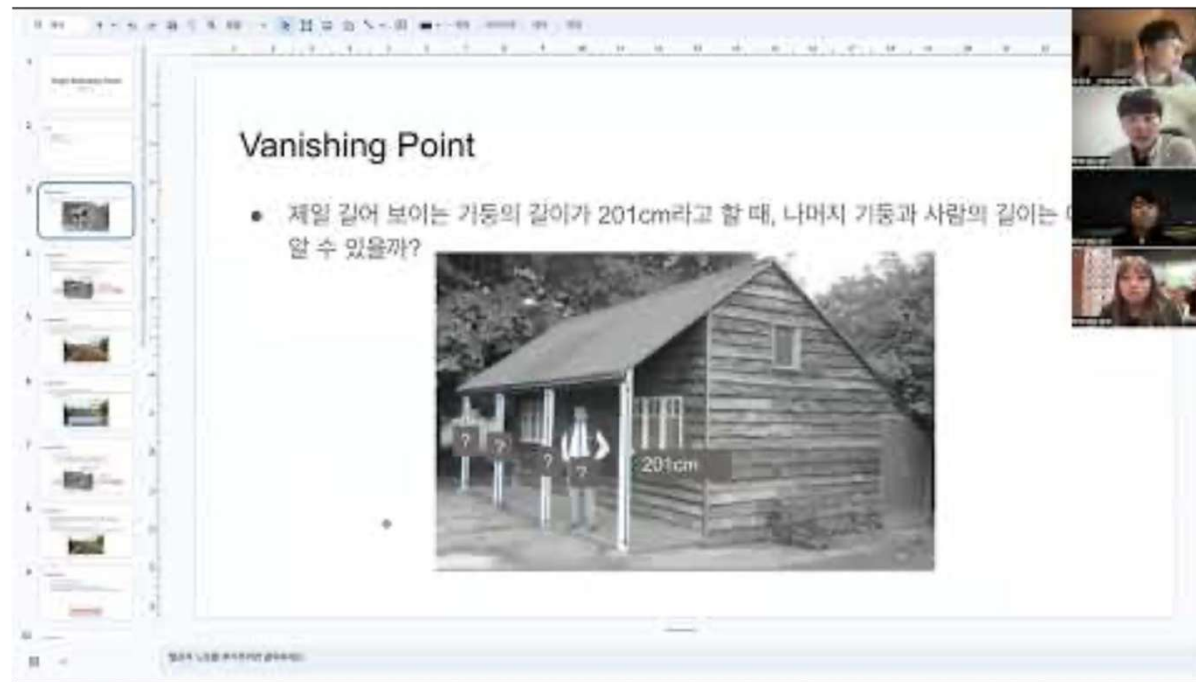
[사진 4] Vanishing Point & Line 1



[사진 5] Vanishing Point & Line 2

■ Demo Video

Single View Geometry 관련 Vanishing Point & Line 내용에 대해 세미나 형식으로 진행했습니다.



[동영상] <https://www.youtube.com/watch?v=-ZYTrtNs5Vk>

CCTV 영상 내 객체 높이 추정

■ 프로젝트 회고

- 현장에서 Reference Line이 될만한 것의 높이를 알 수 있다면 어떻게든 용의자의 키를 특정할 수 있음.
- X, Y, Z 각 방향 별 평행선을 특정할 수 없는 환경이하면 Vanishing Point를 만들 수 없게 되고, 결과적으로 높이 추정이 불가능함.
- 사용자가 GUI를 통해 마우스 클릭을 입력할 때 오차를 줄이기 위한 전처리 작업이 필요함.

■ 성과

- 산학 과제 계약 연장
- 방학 동안 인턴쉽 기회를 얻을 수 있었음.

2024.01 ~ 2024.02 (4인)

한동대학교 교내 CCTV 시스템 구축

■ 프로젝트 개요

Git : https://github.com/InternetOfTough/CCTV_HaLow

목표 : 여러 CCTV노드에서 촬영한 영상의 실시간 웹 스트리밍 서비스 제공

배경 : 유선 연결이 힘든 상황에서의 CCTV 설치를 위해.

주요 기능 : 무선 CCTV 영상 송수신, 실시간 동영상 웹 스트리밍.

■ 담당 역할 및 기여

역할 : 임베디드 프로그래밍, 영상처리, gRPC 통신 개발

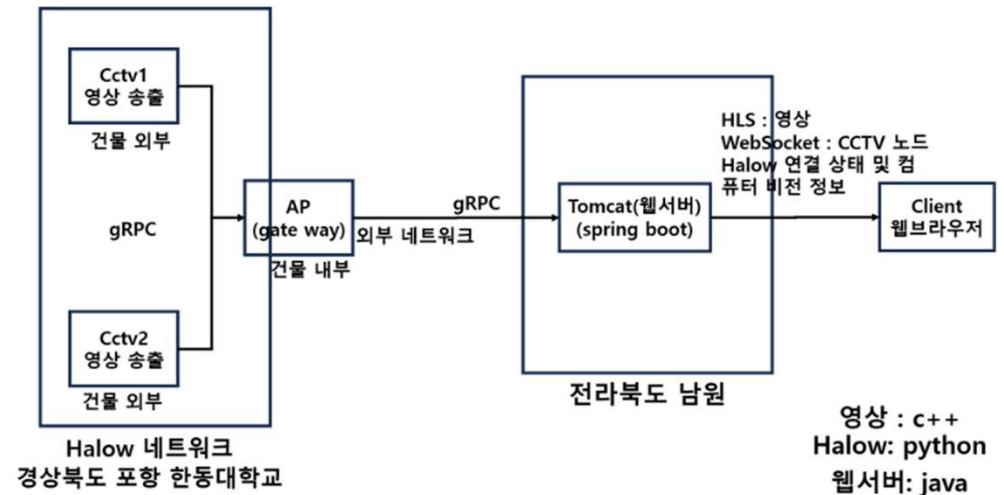
기여 : CCTV 노드 및 게이트웨이 설치. 라즈베리파이에 Wi-Fi HaLow 모듈 설치 및 통신. Crontab으로 CCTV노드 자동 실행 및 에러 처리. gRPC로 영상 송신.

■ 사용 기술

gRPC : Protobuf를 사용하여 보다 효율적이고 빠른 데이터 직렬화 기능을 제공. 이는 영상 데이터를 송신하는데에 있어 일반적인 TCP보다 빠름.

Wi-Fi HaLow : 900MHz의 낮은 대역폭으로 통신속도는 느리지만 그 통신 범위는 대략 900m까지 이른다. 원거리 환경에서 무선 통신을 위해 사용.

Crontab : 리눅스 환경인 CCTV 노드에 이상이 생겼을 시에 대처를 위해 사용.



[사진 1] System Architecture

2023.11 ~ 2023.12 (1인)

자율 주행 경기장 관제 시스템 개발

■ 프로젝트 개요

Git : 보안상 공개 불가

회사 : 자이트론 (인턴)

목표 : 자율 주행 시간 측정 및 미션 확인 여부의 자동화.

배경 : 기존엔 자율주행 경기장의 운용이 모두 수작업으로 이루어졌음.

주요 기능 : 경기 시간 측정, 노드간 ROS2 무선 통신.

■ 담당 역할 및 기여

역할 : 1인 프로젝트

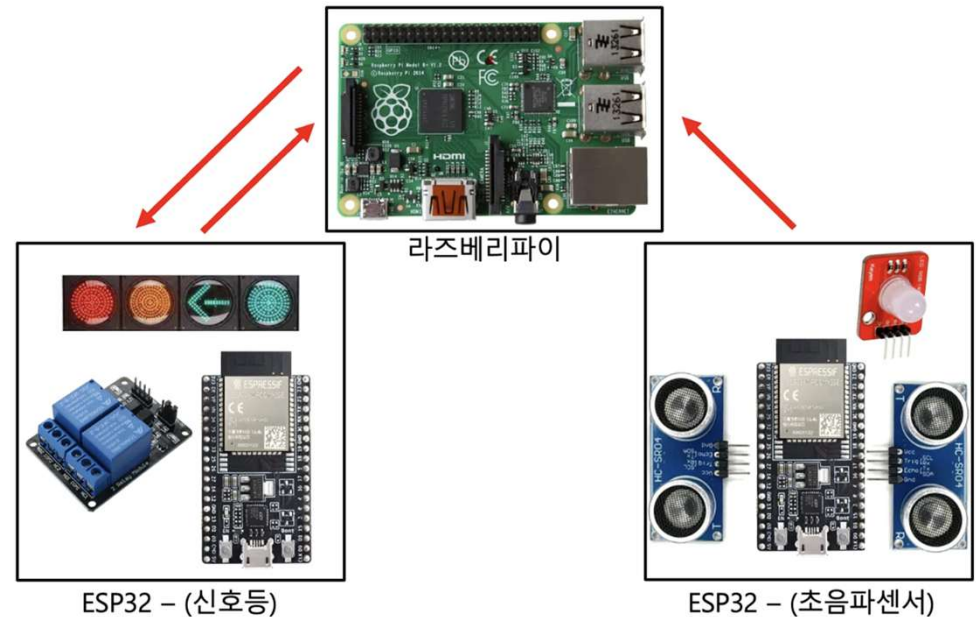
기여 : ROS2 System 구축, PyQT5 GUI 상황판 구현. ESP32 펌웨어 개발.

■ 사용 기술

ROS2 : 기존에 회사에서는 ROS1을 사용했지만 Micro-ROS와의 호환성 때문에 ROS2를 최초로 사용함.

Micro-ROS : 임베디드 보드와 Linux 간의 통신을 위해 사용.

- 라즈베리파이 1대, ESP32 2대를 네트워크 통신 환경에서 이용함



[사진 1] System Architecture

■ 구현 사항

[라즈베리 파이]

- ESP32기기들이 접속하도록 AP Mode로 동작.
- 네트워크에 접속한 기기들과 Micro-ROS Agent 패키지를 이용해 ROS 환경 구축
- ESP32 노드들 간의 통신을 원활하게 하기 위해 GUI 기능 제공.
- 경기의 시작과 미션 점검, 시간 측정을 주관.

[ESP32 초음파 노드]

- 초음파 센서 값을 토픽으로 발행. (Topic Interface)
- 미션 성공 시 부착되어있는 LED 점등과 함께 라즈베리파이로 신호 송신. (Service Interface)

[ESP32 신호등 노드]

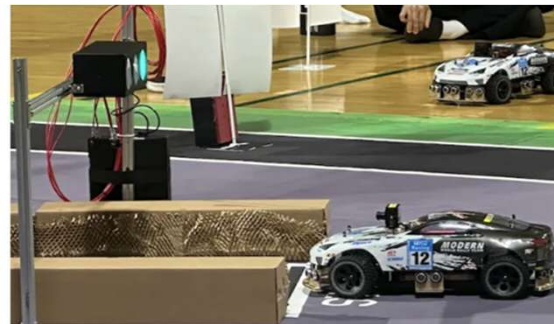
- 라즈베리파이로부터 출발 신호를 수신.
- 녹색등 점등 완료 후 점등 결과 반환. (Service Interface)
- 신호등의 점멸은 릴레이 모듈을 사용하여 전력 공급을 제어하였음.



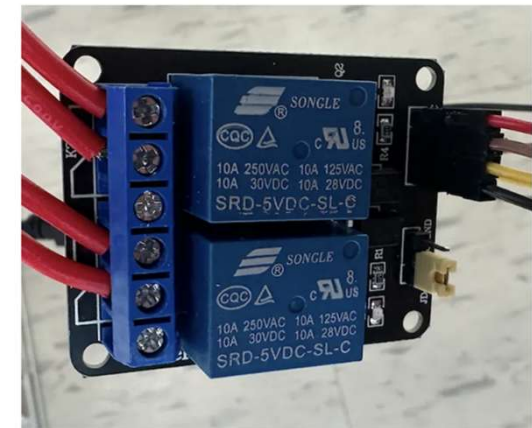
[사진 2] 라즈베리파이 상황판 GUI



[사진 3] 초음파 센서 노드



[사진 4] 신호등 노드



[사진 5] 릴레이 모듈이 설치된 모습

자율주행 경기장 관제 시스템 개발

■ 문제 해결 사례

문제 : PyQT와 ROS2 쓰레드 간의 동기화 문제로 프로그램이 간헐적으로 꺼지는 문제가 발생함.

해결 방법 : PyQT는 무조건 메인 쓰레드에서 동작해야함을 인지하여 기존에 메인 쓰레드에서 돌던 ROS2를 멀티쓰레드에서 동작하도록 변경했음. ROS2는 `spin()` 대기함 수는 대부분 메인쓰레드에서 동작하지만 GUI 혹은 API 서버와 같은 특정한 프로그램과 함께 동작할 때는 멀티 쓰레드로 빼주어야 함을 알게 되었음.

■ 프로젝트 회고

- 멀티 쓰레드를 운용함에 있어 해당 라이브러리의 특성을 이해함이 중요함을 느꼈음.
- 임베디드 보드의 ROS 시스템 운용은 무한한 확장 가능성을 지니고 있으며, 다양한 분야에서 사용이 가능할 것으로 보임.
- ROS1에는 없던 Interface(Service, Action)을 통해 ROS2환경에서 더욱 간편하고 안전한 통신이 가능함.

2023.09 ~ 2023.10 (1인)

자율 주차 프로그램 개발

(Path-Planning & Path-Tracking)

■ 프로젝트 개요

Git : 보안상 공개 불가

회사 : 자이트론 (인턴)

목표 : 카메라를 활용한 차량의 자율 주차.

배경 : 자율 주행 시장의 확대.

주요 기능 : 카메라의 현재 위치 추정, 경로 생성, 경로 추종

■ 담당 역할 및 기여

역할 : 1인 프로젝트

기여 : Camera Calibration, 경로 생성 및 추종 알고리즘 적용. PyGame GUI 구현

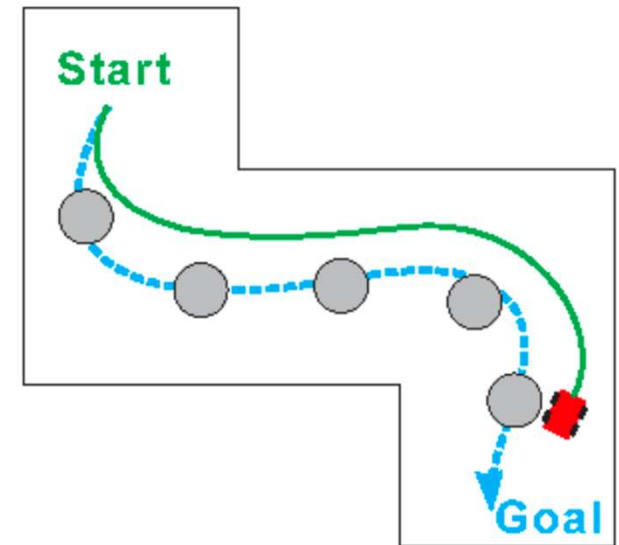
■ 사용 기술

AR-Tag : AR-Tag 기준 카메라의 현재 Pose를 계산

PyGame : 차량의 움직임을 2D 시뮬레이션으로 동기화 하여 구현.

Path-Planning : 차량의 현재 위치로부터 목적지까지의 경로를 생성.

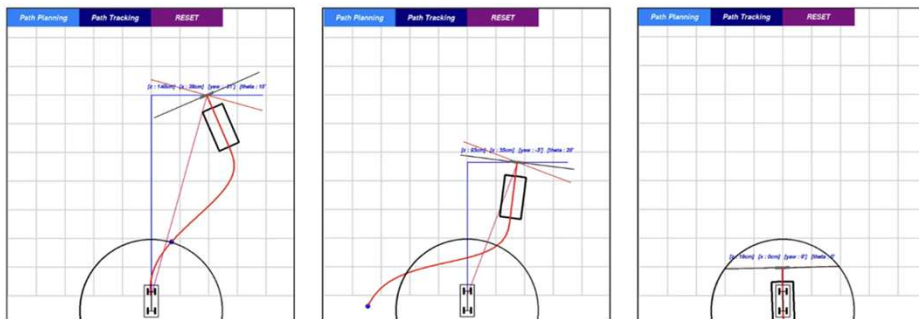
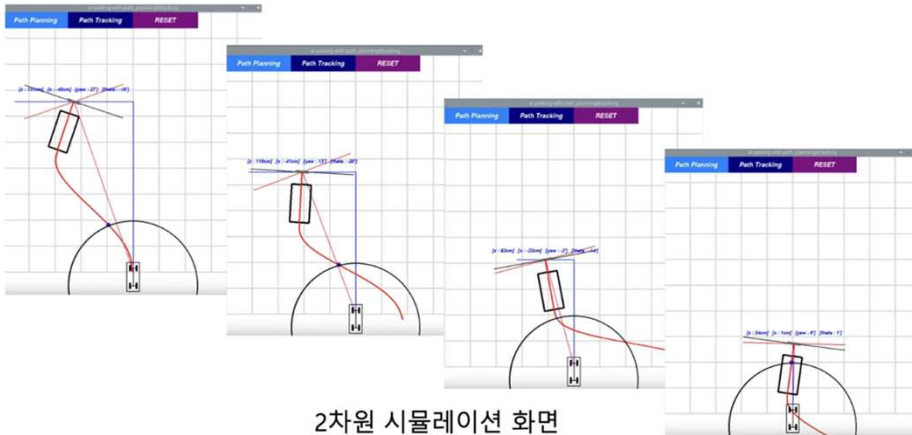
Path-Tracking : 차량의 특성을 고려하여 경로를 따라갈 수 있도록 제어.



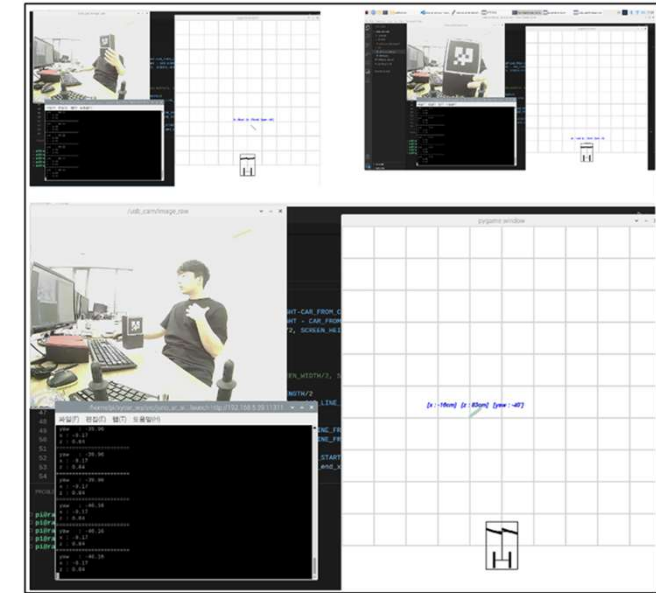
[사진 1] 경로 생성 및 추종

■ 구현 사항

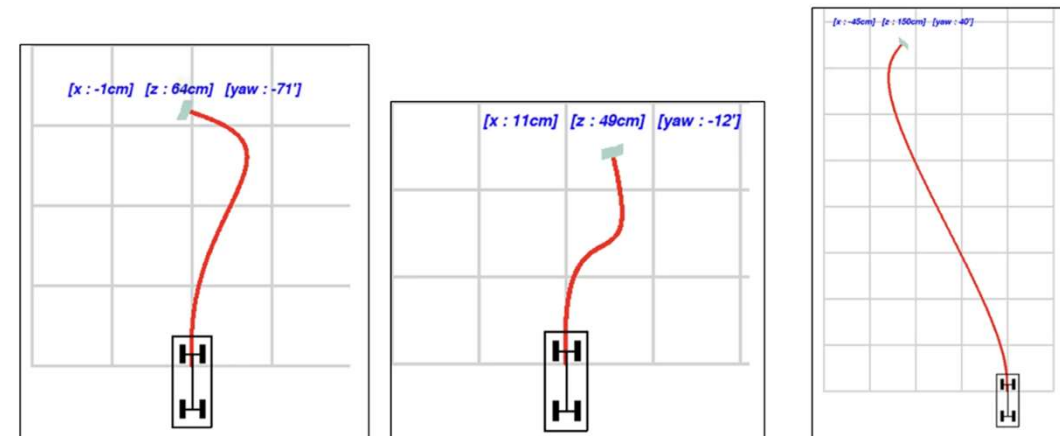
- Camera Calibration으로 알아낸 Intrinsic Parameter를 이용하여 입력 영상의 왜곡 제거.
- 왜곡을 제거한 이미지에서 AR-Tag를 식별해 현재 카메라의 위치를 추정.
- **Quintic-Polynomial** 경로 생성 알고리즘을 이용하여 현재 **Pose**(위치,자세)로부터 도착지점 **Pose**까지의 경로 생성.
- 경로 생성 후 Pure-Pursuit 알고리즘을 이용하여 차량이 경로를 따라가도록 제어.
- 이 모든 과정을 PyGame 라이브러리를 이용하여 시각화 및 제어.



[사진 4] 경로를 Tracking하도록 차량 제어



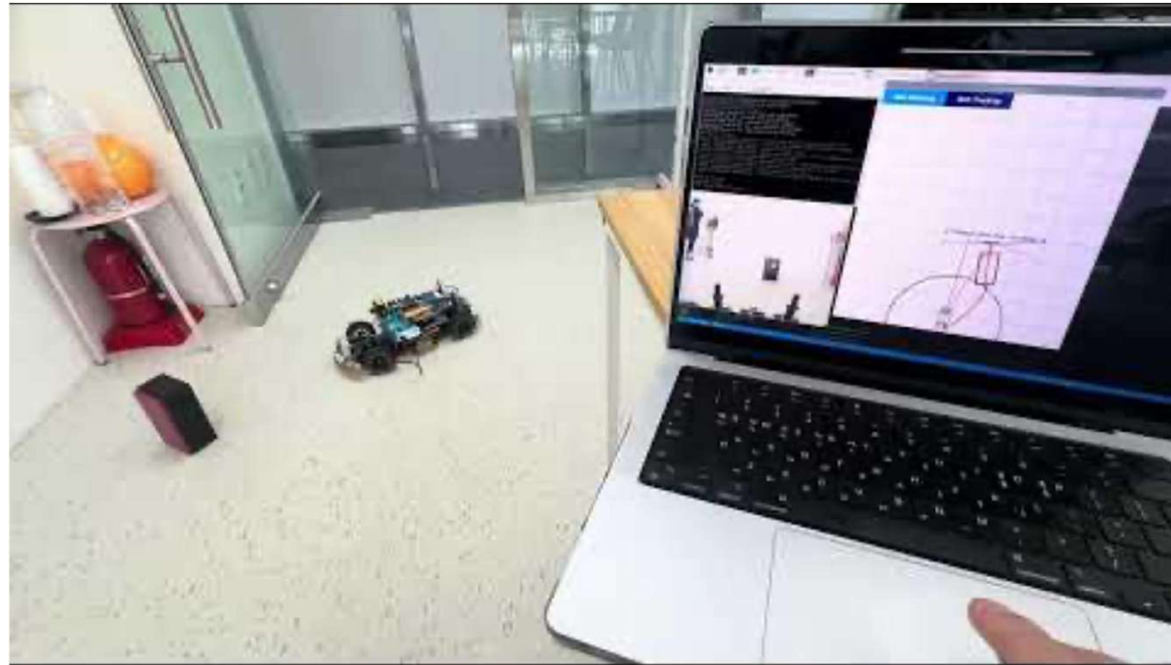
[사진 2] AR-Tag 식별



[사진 3] 차량의 Pose에 따라 다르게 생성되는 경로

자율 주차 프로그램 개발

■ Demo Video



[동영상] <https://www.youtube.com/watch?v=-GKsm9lfCZk>

■ 문제 해결 사례

문제 : AR-Tag를 통한 위치 추정이 제대로 이루어 지지 않고 오차가 5cm 이상 생기는 지점이 생김.

해결 방법 : Camera Calibration 과정에서 입력 이미지가 부족했기 때문이라 판단하여 입력 이미지를 기존 30 ~ 40 장에서 80 ~ 100 장으로 두 배 이상 늘림. Calibration 시간이 1시간 넘게 걸렸지만 기존에 오차가 생기던 증상은 사라졌음.

■ 프로젝트 회고

- 차량의 특성과 주행 환경을 고려한 경로 생성 및 추종 알고리즘 선택이 중요함을 느꼈음.
- 모든 영상 처리 프로젝트의 시작은 Camera Calibration이며 시작이 제일 중요함을 느꼈음.

2023.05 ~ 2023.06 (4인)

딥러닝을 활용한 가상 화폐 자동 매매 프로그램 개발

■ 프로젝트 개요

Git : <https://github.com/YooJuno/Upbit-Auto-Trader>

목표 : 자동화 매매 프로그램으로 사람의 개입 없이 수익 창출.

배경 : 가상화폐 거래 시장의 규모 확대.

주요 기능 : 자동 모델 학습 및 결과 예측을 토대로 가상화폐 매매.

■ 담당 역할 및 기여

역할 : 팀장. 딥러닝 모델 선정 및 학습.

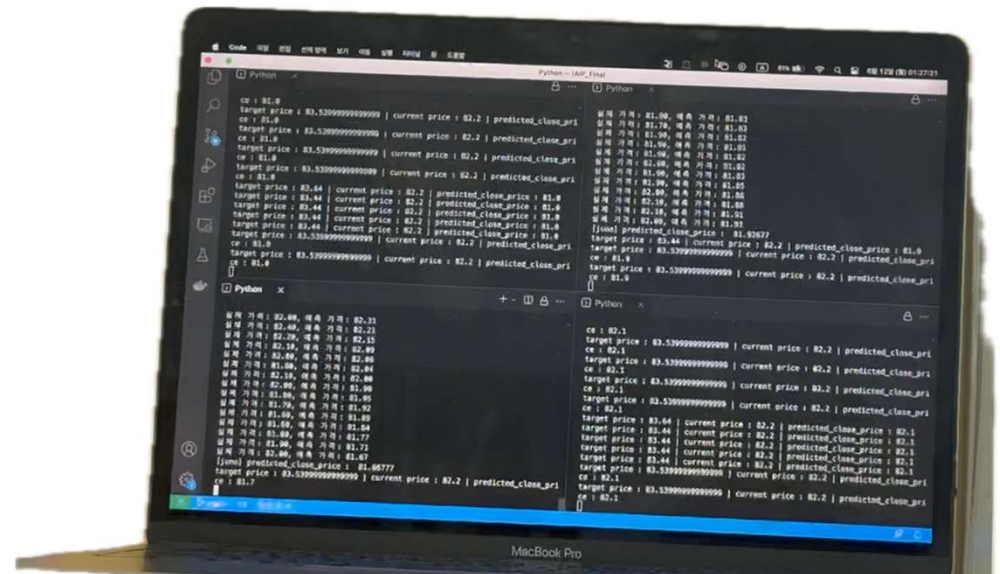
기여 : 주제 제안. 매매조건 및 전략 수립. 매매 프로그램 개발.

■ 사용 기술

LSTM : 시계열 데이터의 예측에 최적화된 딥러닝 모델. 가상 화폐 데이터에 적합.

Upbit API : 가상 화폐를 파이썬 코드로 사고 팔 수 있도록 해주는 API.

변동성 돌파 전략 : 가격 변동폭이 큰 가상 화폐 시장에 적합한 매매 전략.



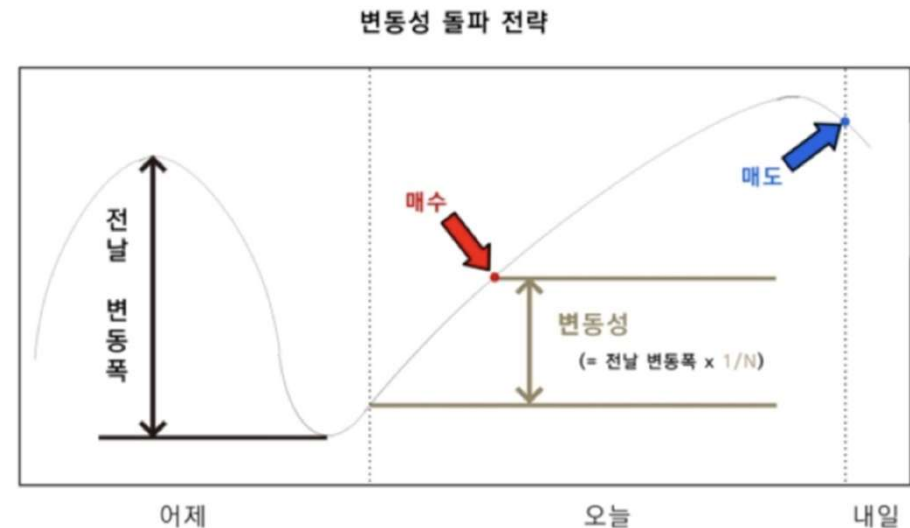
[사진 1] 실시간 LSTM 모델 학습 및 가상화폐 매매

■ 구현 사항

- LSTM 모델을 사용하여 이전에 학습한 데이터를 사용하지 않는 RNN 모델의 단점을 극복.
- 학습 방식을 4가지로 나누어 데이터 종류에 따른 결과를 비교함.
 - 1) 1분봉 데이터 500개 : 이전 500분까지의 데이터
 - 2) 1분봉 데이터 2000개 : 이전 2000분까지의 데이터
 - 3) 10분봉 데이터 500개 : 이전 5000분까지의 데이터
 - 4) 100분봉 데이터 2000개 : 이전 20000분까지의 데이터
 (Train : Test = 9 : 1)
- 변동성 돌파 전략을 적용하여 높은 등락에 대비하였음. (전날의 변동폭에 따라 매매 전략을 세움)
- 10분단위로 모델을 새로 학습함.

```
# 모델 구성
model = tf.keras.models.Sequential([
    tf.keras.layers.LSTM(128, return_sequences=True, input_shape=(seq_length, 1)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1)
])
# 모델 컴파일
model.compile(optimizer='adam', loss='mse')
# 모델 학습
model.fit(X_train, y_train, epochs=50, batch_size=32, shuffle=False)
# 예측 수행
y_pred = model.predict(X_test)
# 스케일 역변환
y_pred = scaler.inverse_transform(y_pred)
y_test = scaler.inverse_transform(y_test)
```

[사진 2] LSTM 모델 구성



[사진 3] 변동성 돌파 전략

딥러닝을 활용한 가상 화폐 자동 매매 프로그램 개발

■ 결과

- 리플은 이익을 봤고 도지는 손해를 보았다.
- 변동성 돌파 전략은 변동성이 클 수록 이익을 보는 특성을 지녔기 때문에 변동성이 더 큰 리플이 도지에 비해 이익을 볼 수 있었던 것으로 보인다.

■ 프로젝트 회고

- 암호화폐와 매매전략의 특성으로 높은 수익률을 기대하긴 힘들다.
- 해당 종목의 호재 및 악재 반영이 힘들다.
- 종목마다 특성 차이가 명확하다.
- 암호화폐의 특성상 종가가 없기 때문에 변동성 돌파 전략을 적용하는 데에 한계가 따른다.

도지 코인

- 1분봉 500개 (-0.974%)

내 보유자산	KRW 환산 추정값
보유KRW 10,227	총 보유자산 10,227
총매수 0	평가손익 수익률 -0.974%
총평가 0	

제결시간	코인명	마켓	종류	거래수량
23.06.12 22:49	DOGE	KRW	매도	127.43123289 DO
23.06.12 07:16	DOGE	KRW	매수	127.36737591 DO
23.06.12 07:09	DOGE	KRW	매도	127.65009913 DO
23.06.12 07:08	DOGE	KRW	매수	127.64995736 DO
23.06.12 07:00	DOGE	KRW	매도	127.93349637 DO
23.06.12 07:00	DOGE	KRW	매수	127.99749512 DO

- 1분봉 2000개 (1.83%)

내 보유자산	KRW 환산 추정값
보유KRW 9,756	총 보유자산 9,760
총매수 5	평가손익 수익률 -1.83%
총평가 4	

제결시간	코인명	마켓	종류	거래수량
23.06.12 23:05	DOGE	KRW	매도	121.55934704 DOGE
23.06.12 07:33	DOGE	KRW	매수	121.55928623 DOGE
23.06.12 07:30	DOGE	KRW	매도	121.68090742 DOGE
23.06.12 07:25	DOGE	KRW	매수	121.74177831 DOGE

10분봉 2000개 (-1.1%)

내 보유자산	KRW 환산 추정값
보유KRW 9,971	총 보유자산 9,976
총매수 6	평가손익 수익률 -1.10%
총평가 5	

제결시간	코인명	마켓	종류	거래수량
23.06.12 23:04	DOGE	KRW	매도	123.38569966 DOGE
23.06.12 23:04	DOGE	KRW	매도	0.69400000 DOGE
23.06.12 08:35	DOGE	KRW	매수	124.14177055 DOGE

10분봉 500개 (-1.47%)

내 보유자산	KRW 환산 추정값
보유KRW 9,729	총 보유자산 9,734
총매수 5	평가손익 수익률 -1.47%
총평가 4	

제결시간	코인명	마켓	종류	거래수량
23.06.12 23:04	DOGE	KRW	매도	121.22190140 DO
23.06.12 07:38	DOGE	KRW	매수	121.22176650 DO
23.06.12 07:38	DOGE	KRW	매도	121.49157085 DO
23.06.12 07:38	DOGE	KRW	매수	121.49173227 DO

[사진 4] 도지 코인 결과



- 도지코인 일봉



- 리플 일봉

리플

- 1분봉 500개 (+0.29%)

리플 (XRP)	평가손익 수익률	30 0.29%
14.79377771 XRP 보유수량	691 KRW 매수평균가	
10,252 KRW 평가금액	10,223 KRW 매수금액	

- 1분봉 2000개 (+0.14%)

리플 (XRP)	평가손익 수익률	14 0.14%
14.11209739 XRP 보유수량	691 KRW 매수평균가	
9,765 KRW 평가금액	9,752 KRW 매수금액	

- 10분봉 500개 (+0.43%)

리플 (XRP)	평가손익 수익률	42 0.43%
14.09331797 XRP 보유수량	690 KRW 매수평균가	
9,766 KRW 평가금액	9,725 KRW 매수금액	

[사진 5] 리플 코인 결과

[사진 6] 도지, 리플 변동성 비교

딥러닝을 활용한 가상 화폐 자동 매매 프로그램 개발

2023.05 ~ 2023.06 (2인)

Boost ASIO를 활용한 Online Pacman 게임 개발

■ 프로젝트 개요

Git : <https://github.com/YooJuno/PacmanGame-BoostASIO>

목표 : 온라인 기반 멀티 플레이어 게임 제작

배경 : 서버 비동기 처리 방식의 수요 증가

주요 기능 : 실시간 클라이언트-서버 통신을 통해 다중 사용자가 Pacman을 조작하고 점수를 획득하는 인터페이스 기반 온라인 게임 구현

■ 담당 역할 및 기여

역할 : Pacman 게임 인터페이스 제작. 클라이언트 비동기 통신 설계.

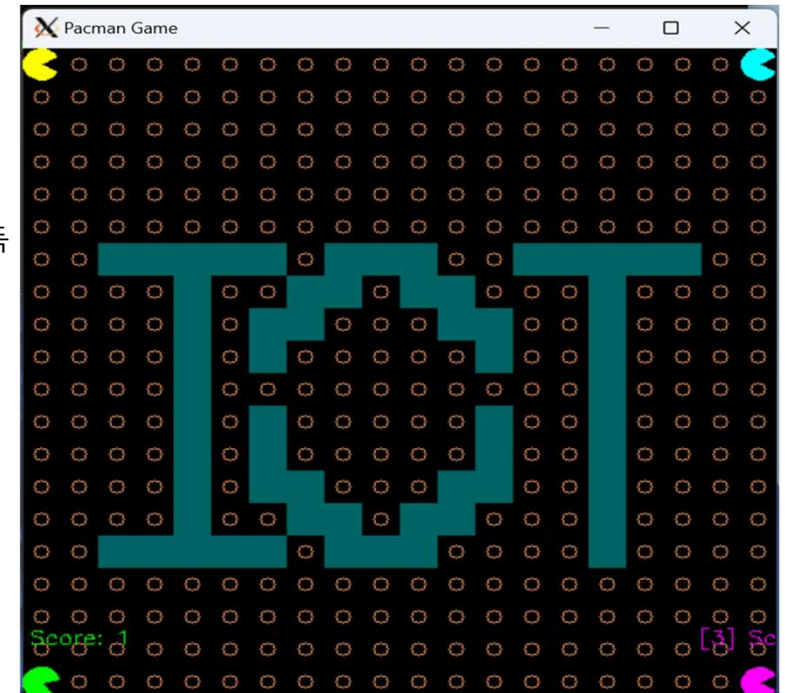
기여

- 클라이언트 입력을 서버에 전송하고, 서버의 상태 응답을 받아 UI에 실시간 반영하는 구조 설계
- Boost ASIO 기반 io_context, TCP 소켓, 핸들러 스트랜드를 활용한 안전한 동시 처리 구조 적용
- OpenCV를 이용한 화면 렌더링 (Pacman 움직임, 점수 표시 등)

■ 사용 기술

Boost ASIO : 비동기 서버-클라이언트 프로그램 개발을 위해 사용.

OpenCV : Pacman 인터페이스 개발을 위해 사용.



[사진 1] Pacman Online Game

■ 구현 사항

[서버]

- io_context 기반 이벤트 루프 구성
- 각 클라이언트별 Session 클래스 관리 → 메시지 송수신, 연결 유지
- 비동기로 들어오는 키 입력을 처리하고, 게임 상태 브로드캐스트 기능 포함

[클라이언트]

- 키 입력 (방향키 등)을 감지하고 서버 전송
- 서버 응답에 따라 OpenCV로 UI 업데이트 (Pacman 위치, 점수 등 그래픽 출력)

[OpenCV UI]

- 2D 그래픽 기반 간단한 UI
- 마우스/키 이벤트 처리 및 점수 정보 출력

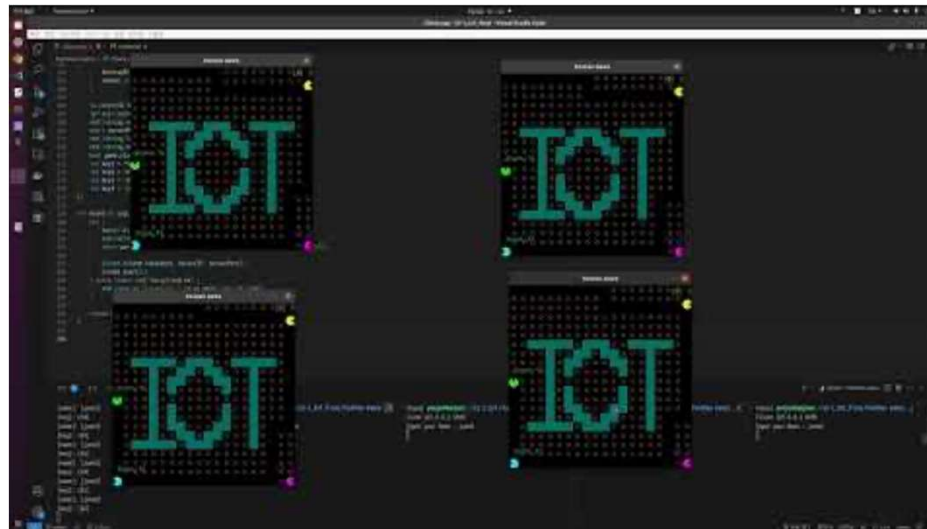
■ 주요 성과

- 비동기 네트워크 통신 성공: Boost ASIO 기반 비동기 TCP 통신을 통해 실시간 클라이언트-서버 간 상태 동기화 구현 [Velog](#)[Seek constantly+2Cafe](#)[M0ca+2GitHub](#)
- 멀티 클라이언트 지원 구조 설계: 여러 클라이언트 세션을 개별적으로 관리하며 동시 접속 처리 가능
- 클라이언트-서버 로직 분리: UI와 네트워크 로직을 모듈화하여 유지보수성 향상

■ 한계점

- 네트워크 최적화 부족: 전송되는 메시지 패킷의 효율적 직렬화/압축 미비 → 대량 클라이언트 처리시 지연 발생 우려
- 게임 로직 단순화: 현재는 단일 맵 중심의 이동 및 점수 처리만 구현 → Ghost AI, 충돌, 레벨 등 자동화 기능 미포함
- 멀티스레딩 한계: strand만으로 동기화 처리. IOCP 기반 고성능 확장성 고려되지 않음 Seek constantly
- OpenCV UI 제약: 그래픽 성능 및 애니메이션 기능이 제한적. OpenGL 또는 SDL 기반 고성능 UI로의 확장 가능

■ Demo Video



[동영상]

<https://www.youtube.com/watch?v=2uh6XcFZric>

Boost ASIO를 활용한 Online Pacman 게임 개발

2023.04 ~ 2023.06 (4인)

STM32기반 LED피아노 제작 (DMA + SPI)

■ 프로젝트 개요

Git : <https://github.com/YooJuno/STM32-MBED/tree/main/Final>

목표 : 임베디드 피아노 모듈 개발.

주요 기능 : 딜레이가 없는 피아노 건반 및 스피커와 LED 기능 구현.

■ 담당 역할 및 기여

역할 : 피아노 회로 제작 및 DMA+SPI 통신 개발.

기여 : LED모듈 타이밍에 맞는 DMA 타이밍 제어 및 Bit Map Setup.

■ 사용 기술

RTOS : mbed OS를 사용하여 STM32 펌웨어 개발.

SPI : LED 모듈 제어

DMA : LED 모듈 제어 시 딜레이를 최소화 하기 위해.

PWM : 해당 피아노 버튼에 해당하는 음 출력.

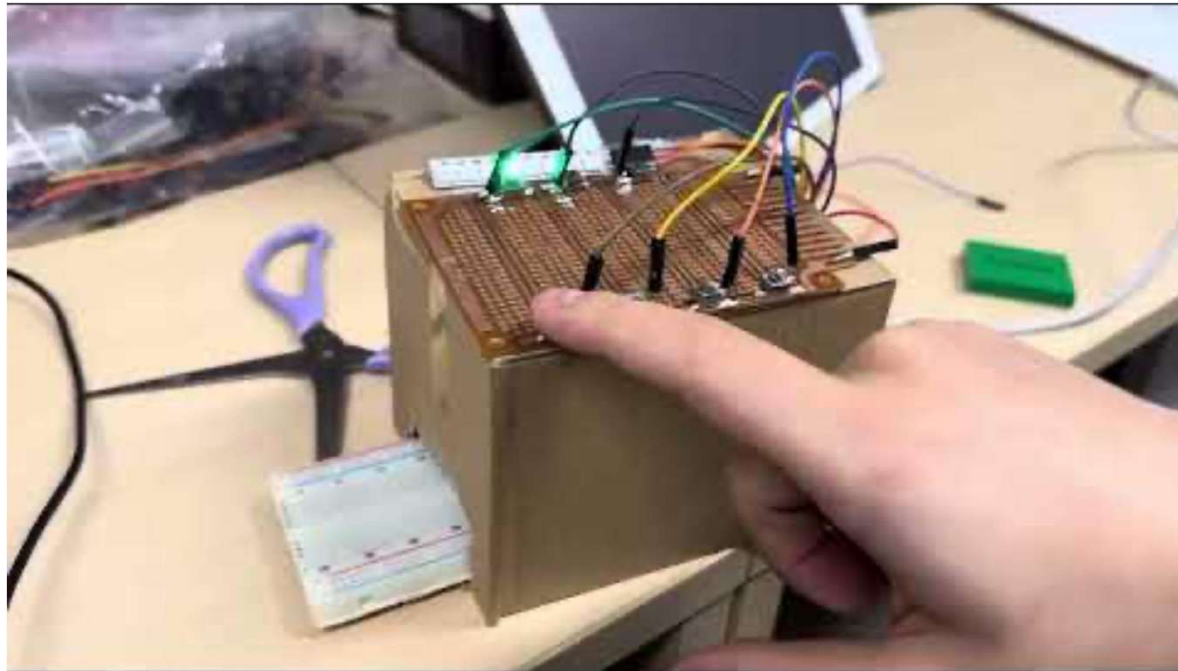
■ 구현 사항

- 피아노 건반을 누르면 LED모듈이 해당하는 색으로 점등되고 피에조 모듈에서 소리가 출력됨.
- 기존 SPI 통신은 CPU Bus를 통하기 때문에 딜레이가 큼. 이를 DMA방식과 결합하여 Real Time 통신을 가능하도록 함.



[사진 1] System Architecture

■ Demo Video



[동영상] <https://www.youtube.com/watch?v=sW-dLjTRV7A>

2022.07 ~ 2023.06 (5인)

PilotNet 성능 향상을 위한 SLAM과 YOLO 활용

■ 프로젝트 개요

Git : https://github.com/YooJuno/Autonomous-Driving_E2E_SLAM

논문 : <https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11522374>

회사 : 앤씨앤 (기술 자문)

목표 : 자율주행 모델인 PilotNet의 단점 보완 및 성능 향상

배경 : 자율주행 기술의 관심도 상승 및 시장 확대.

주요 기능 : 경로 학습 및 자율 주행, 객체 인식, 현재 위치 추정 및 지도 작성.

■ 담당 역할 및 기여

역할 : Server 기반 Visual-SLAM 개발 및 YOLO, PilotNet 결합.

기여 : 기존에 차량용 노트북에서 모두 실행하던 구조에서 서버-클라이언트 구조로 변경하여 차량에서의 부하를 줄임. 이를 통해 실시간 자율 주행 가능.

■ 사용 기술

PilotNet : End-To-End Driving 자율주행 딥러닝 모델 (CNN기반)

Visual-SLAM : ORB-SLAM3 사용하여 실시간 위치 추정 수행.

YOLOv5 : 장애물 및 보행자를 인식하여 회피주행.

TCP : 차량의 전방 영상을 서버로 송신하여 리소스 분배.

■ 자율 주행

실시간 자율주행을 위해

SLAM (server)와

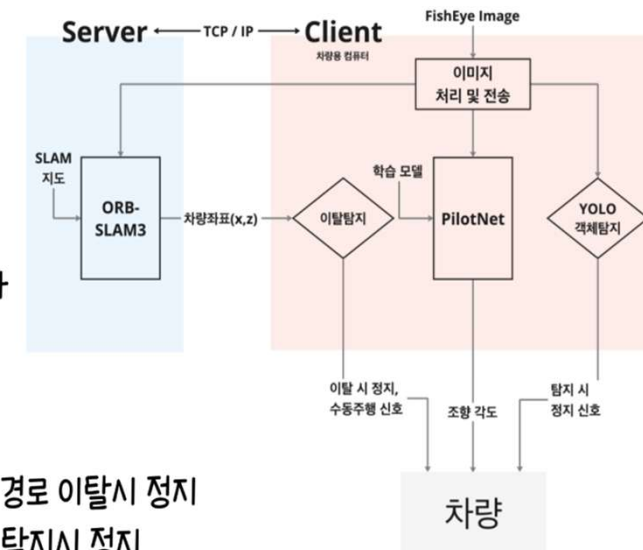
PilotNet, YOLO (client)가

TCP/IP로 통신

◦ PilotNet : 조향각 예측

◦ ORB-SLAM3 : 지정된 경로 이탈시 정지

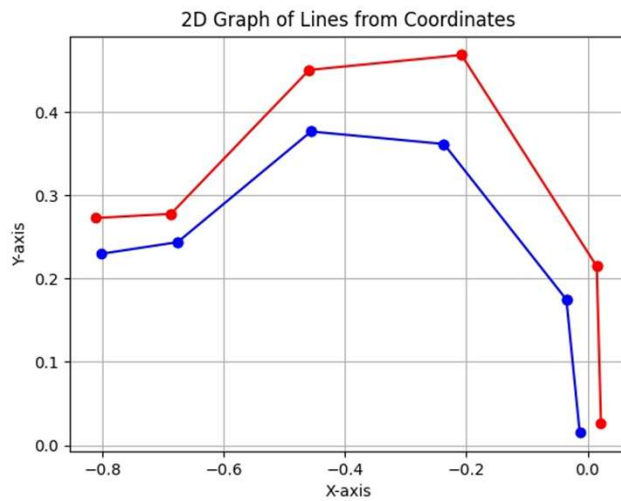
◦ YOLOv4-tiny : 보행자 탐지시 정지



[사진 1] System Architecture

■ 구현 사항

- 정확한 SLAM 처리를 위한 어안렌즈의 Calibration
- PilotNet을 사용하여 경로 학습 및 모델 생성.
- ORB-SLAM3를 사용하여 경로 지도 생성(Mapping) 및 실시간 위치 추정(Localization).
- 생성한 지도를 불러와 현재 위치를 재파악 (Relocalization)
- 차량쪽에서 촬영한 이미지를 실시간으로 서버컴퓨터로 전송하여 Visual-SLAM 수행 후 좌표 값을 차량으로 회신.



[사진 2] SLAM을 통해 생성한 2D 주행 경로 지도

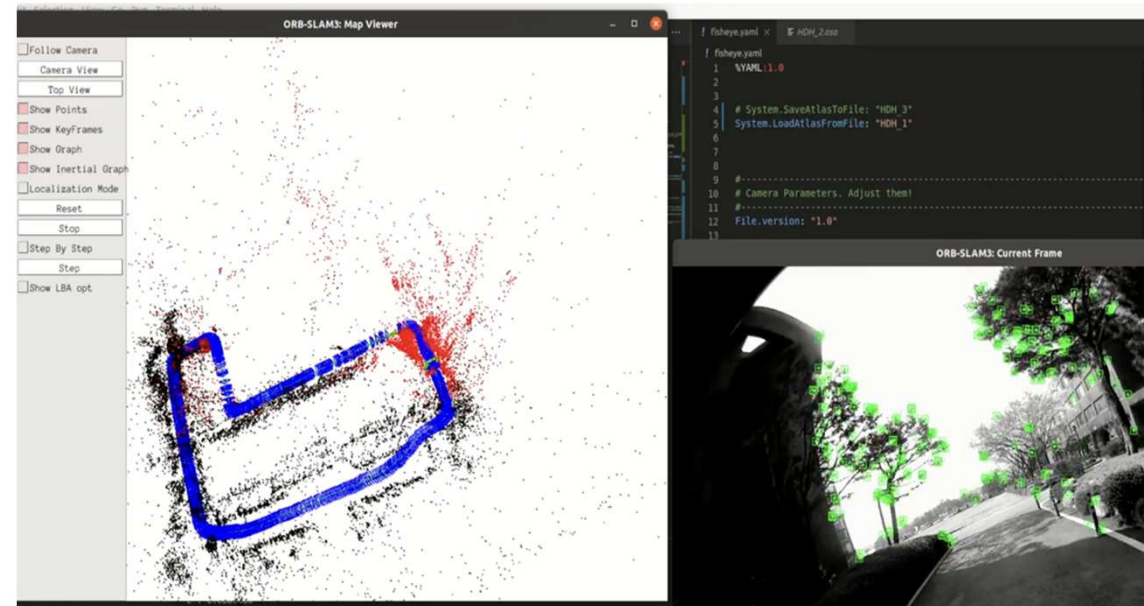


[사진 3] 실제 주행에 사용된 전동차

PilotNet 성능 향상을 위한 SLAM과 YOLO 활용

■ SLAM Map 생성

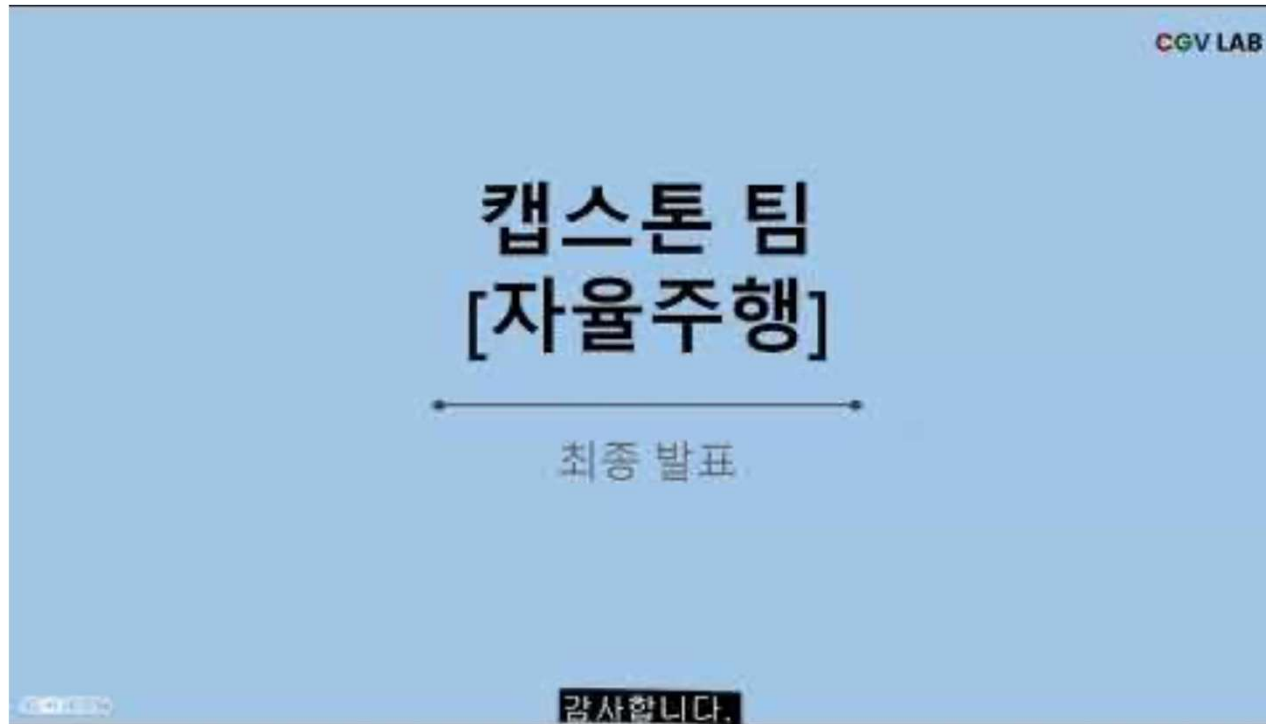
- 3차원 점들의 집합인 Point-Cloud를 생성하여 저장.
- 이후 주행 시에 맵을 불러와 현재 위치 파악 및 추적(Relocalization)



[사진 2] ORB-SLAM3을 통해 생성한 3D Map

PilotNet 성능 향상을 위한 SLAM과 YOLO 활용

■ Demo Video



[동영상 재생목록]

https://www.youtube.com/playlist?list=PLg3p47m-psINqF_8QVGMixhW7zAv6Ir2F

2022.09 ~ 2023.02 (4인)

파노라마 카메라를 활용한 공장 내 작업자 위치 추정

■ 프로젝트 개요

Git : 보안 상 공개 불가

회사 : 포항산업과학기술원 (산학 과제)

목표 : 영상 데이터를 이용하여 작업자의 실시간 이동 경로 확인.

배경 : 공장 규모 확대를 위한 작업자 관리의 필요성.

주요 기능 : 실시간 이동 경로 시각화. 공장 데이터 사전 학습.

■ 담당 역할 및 기여

역할 : Feature Extraction, Matching(SIFT) 알고리즘 최적화.

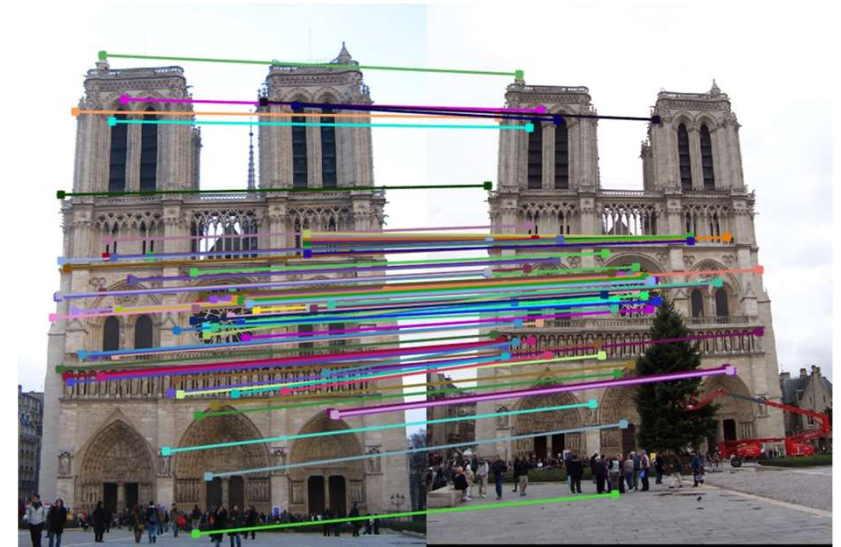
기여 : Radius Matching 알고리즘 개발을 통한 노이즈 감소

■ 사용 기술

Feature Extraction : SIFT Feature 사용

Feature Matching : 이미지 유사도 비교를 위해 사용.

RANSAC : Noise를 제거해 더욱 정확한 Feature Matching을 위해 사용.



[사진 1] Feature Matcing 예시

■ 구현 사항

- 작업자 안전모에 부착한 360° 파노라마 카메라에서 천장 및 전면 이미지를 활용하여 작업자의 위치를 추정.
- 작업자의 위치를 추정하기 위해 이미지 간의 Feature Matching 알고리즘 사용
- 현재 입력 이미지와 이전 입력 이미지를 비교(Matching)하여 특징점이 움직인 거리 벡터를 계산하여 카메라의 이동 경로 계산
- 이동 중 노이즈로 인한 오차를 보완하기 위해 사전에 촬영해둔 DB 이미지와 현재 이미지를 비교하여 현재 위치 업데이트



[사진 2] Visual Odometry 예시

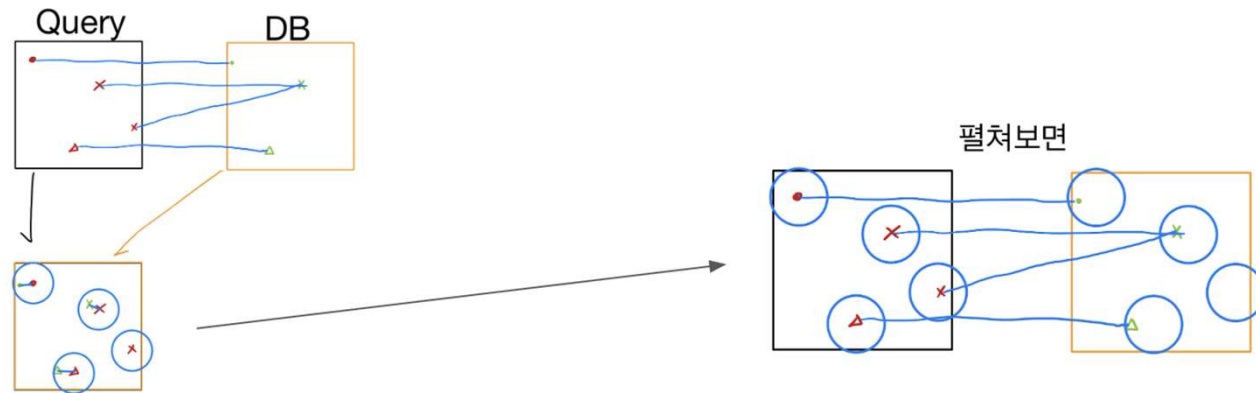
파노라마 카메라를 활용한 공장 내 작업자 위치 추정

■ 문제 사례

- 공장의 특성 상 빛이 적고 주변 모습들이 비슷해 Feature Matching이 제대로 이루어 지지 않음
- 즉, 실제 동일한 위치의 Feature가 아닌 엉뚱한 곳의 Feature가 많이 Matching되는 문제가 지속적으로 발생함.

■ 해결

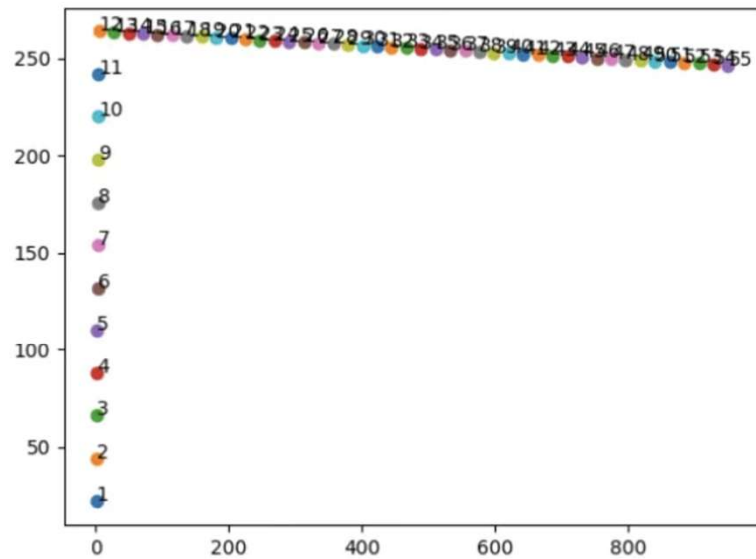
- 작업자의 이동속도가 빠르지 않으며, 그 카메라의 입력 이미지 간의 차이가 크지 않음.
- 이를 이용하여 이전 이미지에서 추출한 Feature 위치의 특정 반경 안에서만 현재 이미지의 Feature를 추출하는 Radius Matching 방식을 고안하여 적용함
- 무분별하게 매칭되던 매칭쌍들이 급격하게 줄어들었으며, 유의미한 결과만 남게 되었음.



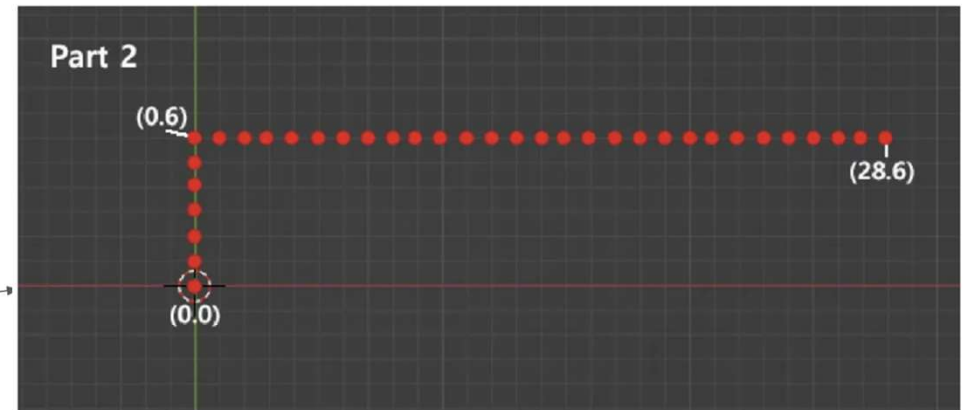
[사진 3] Radius Matching 예시

■ 결과 및 성과

- 공장 내 특정 구간에 대해 실시간으로 추정한 작업자의 위치가 실제 경로와 비슷한 양상을 보임
- 산학 과제 재계약 달성.



< Tracking >



< GT >

파노라마 카메라를 활용한 공장 내 작업자 위치 추정

2021.11 ~ 2021.12 (1인)

영상 처리 기반 자율 주행 드론

■ 프로젝트 개요

목표 : 객체를 인식하여 객체 앞으로 자율 주행.

주요 기능 : 객체 인식. PID 제어. 영상 처리.

■ 담당 역할 및 기여

역할 : 객체 인식 및 제어 최적화.

기여 : 객체를 정확히 인식하여 객체 앞으로 자연스럽게 주행하였음.

■ 사용 기술

OpenCV : HSV 변환 및 Edge 검출

UDP : 원격으로 드론을 조작.

PID : 객체 앞 까지 주행하기 위한 제어 알고리즘.

Djitellopy : 파이썬 텔로 드론 라이브러리

YOLO : 객체를 인식하여 회피 주행을 가능하게 함.



[사진 1] 프로젝트에서 사용한 Tello 드론

2021.07 ~ 2021.08 (3인)

모형차 자율 주행 경진대회

■ 프로젝트 개요

주관 : 자이트론

목표 : 카메라 영상 데이터를 활용한 트랙 자율 주행.

주요 기능 : 차선 인식. 자율 주행. 객체 인식. 회피 주행.

■ 담당 역할 및 기여

역할 : 조향 알고리즘 최적화.

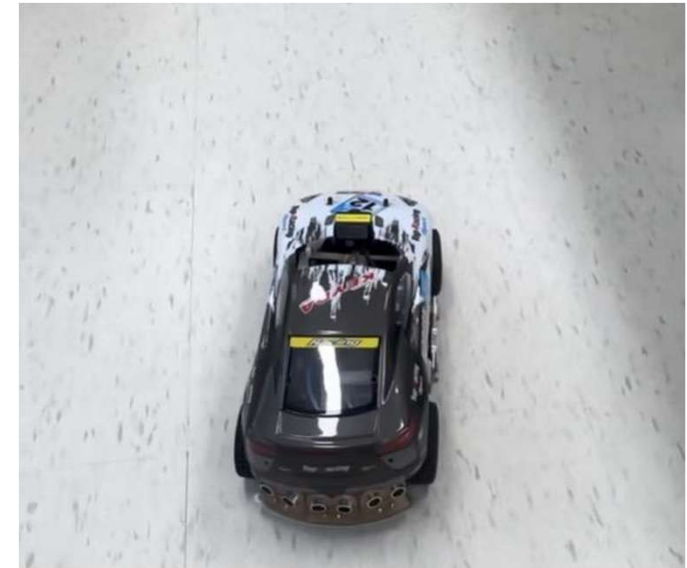
기여 : 부드러운 조향을 가능케 하여 주어진 트랙을 성공적으로 주행.

■ 사용 기술

OpenCV : Hough를 통해 차선을 인식.

PID : 직선과 코너링에서 차선 중앙 주행을 가능하게 함.

YOLO : 객체를 인식하여 회피 주행을 가능하게 함.



[사진 1] 모형차 실사진